

Lecture 6: Pre-training



Intro to Pre-training and Prompting

Data Scaling Laws

Model Scaling Laws

Chinchilla: Data x Model

Putting it together

Administrative details and recap

Announcements

- ▶ Assignment 2 due
- ▶ Assignment 3 released
- ▶ No class next week (“legislative Monday”)
- ▶ Midterm October 28

Intro to Pre-training and Prompting

Data Scaling Laws

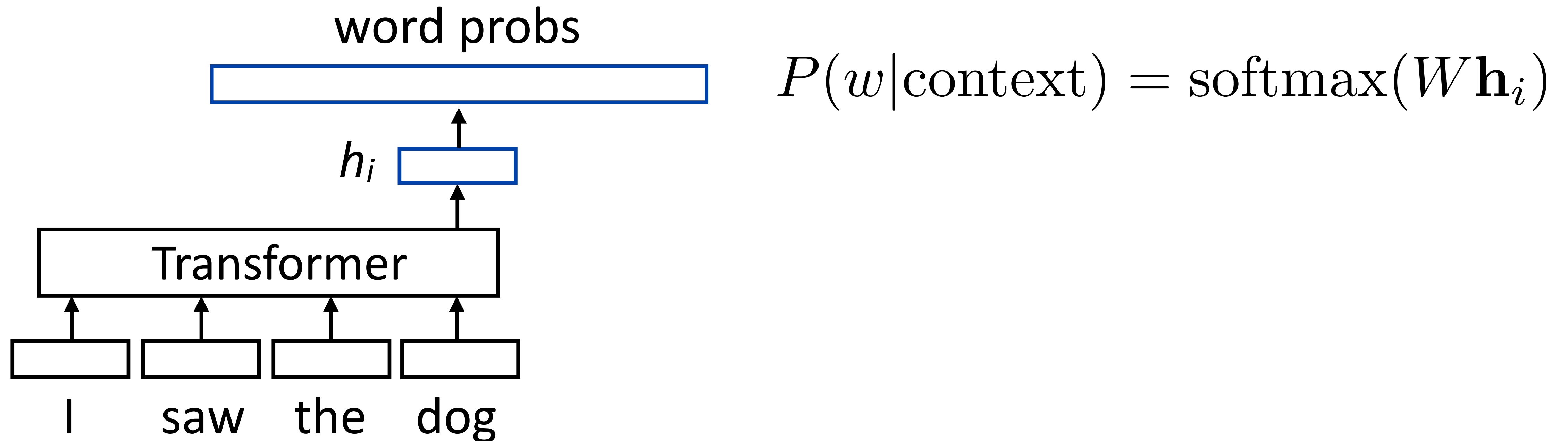
Model Scaling Laws

Chinchilla: Data x Model

Putting it together

Intro to Pretraining and Prompting

Recap: Transformer LMs



We're going to train LLMs on a ton of web data to do this next word prediction task.

Why are such Transformers useful for us? What can they do?

Pre-LLMs: Fine-tuning

- ▶ Fine-tuning: this is the “normal way” of doing learning in ML
- ▶ Requires computing the gradient and applying a parameter update on every example



Brown et al. (2020)

Prompting

- ▶ GPT-3 proposed an alternative: **in-context learning**.

Key idea: a strong LLM should be able to complete the following pattern correctly. What's the most likely next token? The French translation of *cheese*

```
1 Translate English to French:
2 sea otter => loutre de mer
3 peppermint => menthe poivrée
4 plush girafe => girafe peluche
5 cheese => .....
```

← task description

← examples

← prompt

Prompting in general

- ▶ Prompting is **giving an LLM a detailed instruction to perform a task**. This could be a long instruction in natural language, in-context examples, or a combination.
- ▶ If a language model is very good, the most likely continuation of the prompt should be the answer that we want*

**This is not really true. Models are extensively post-trained via supervised fine-tuning and reinforcement learning to answer questions and do tasks we care about*

- ▶ Today: focus on how we get models that are large enough and trained enough to do this effectively. This involves **big models** and **big data**.

Prompting Best Practices

- ▶ **Be specific:** instructions should be clear, ICL examples should articulate edge cases
- ▶ Be specific: instructions should be clear, ICL examples should articulate edge cases
- ▶ **Practices are constantly changing.** We are not going to teach a course on prompting. Models are constantly trained to be more helpful, so tricks like saying “you are an expert mathematician” become less necessary over time.

Intro to Pre-training and Prompting

Data Scaling Laws

Model Scaling Laws

Chinchilla: Data x Model

Putting it together

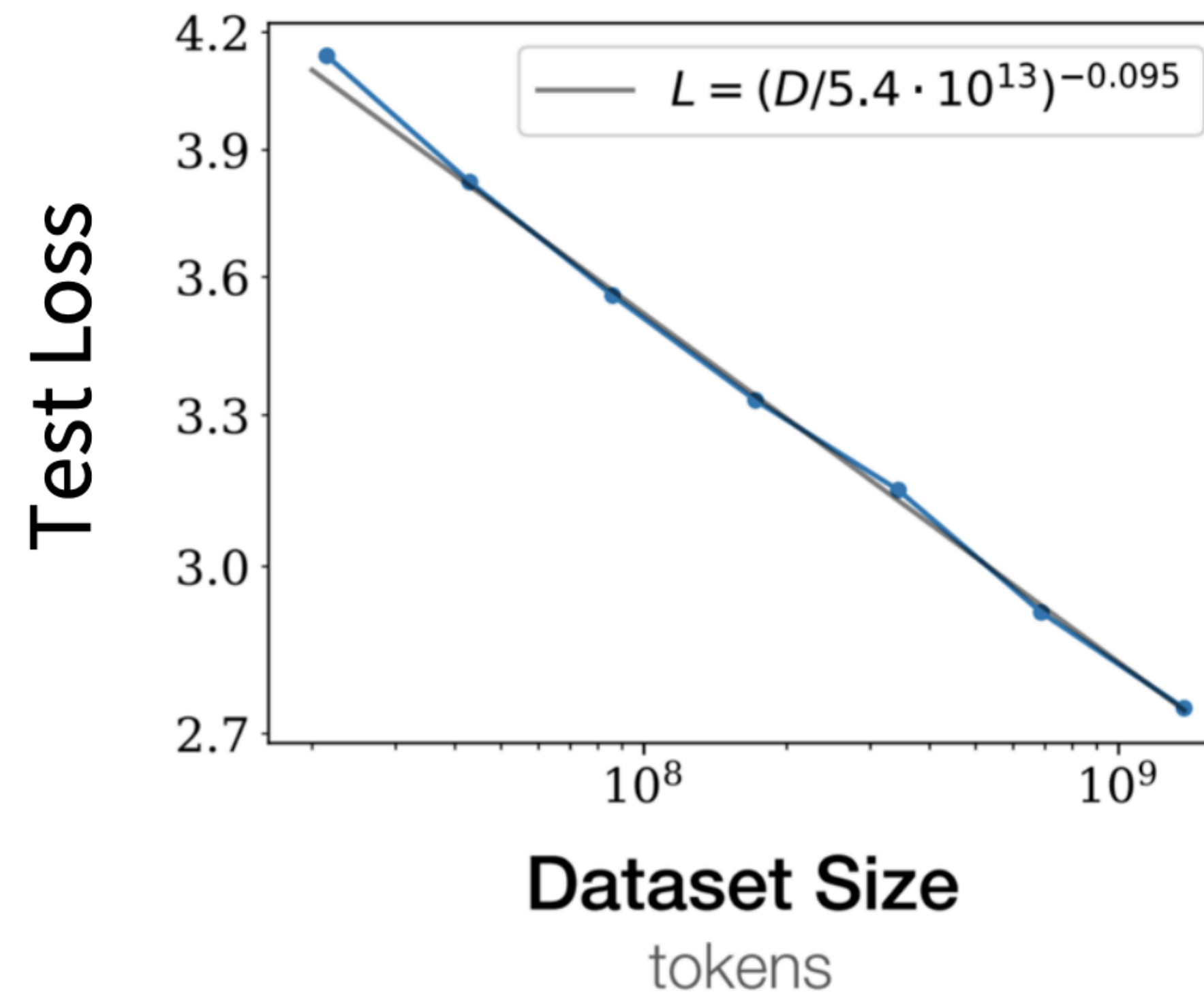
Data Scaling Laws

Slide credit: Tatsu Hashimoto

Data Scaling Laws

First, an empirical observation

Loss and dataset size is linear on a log-log plot



“Scale-free” or
“Power law”

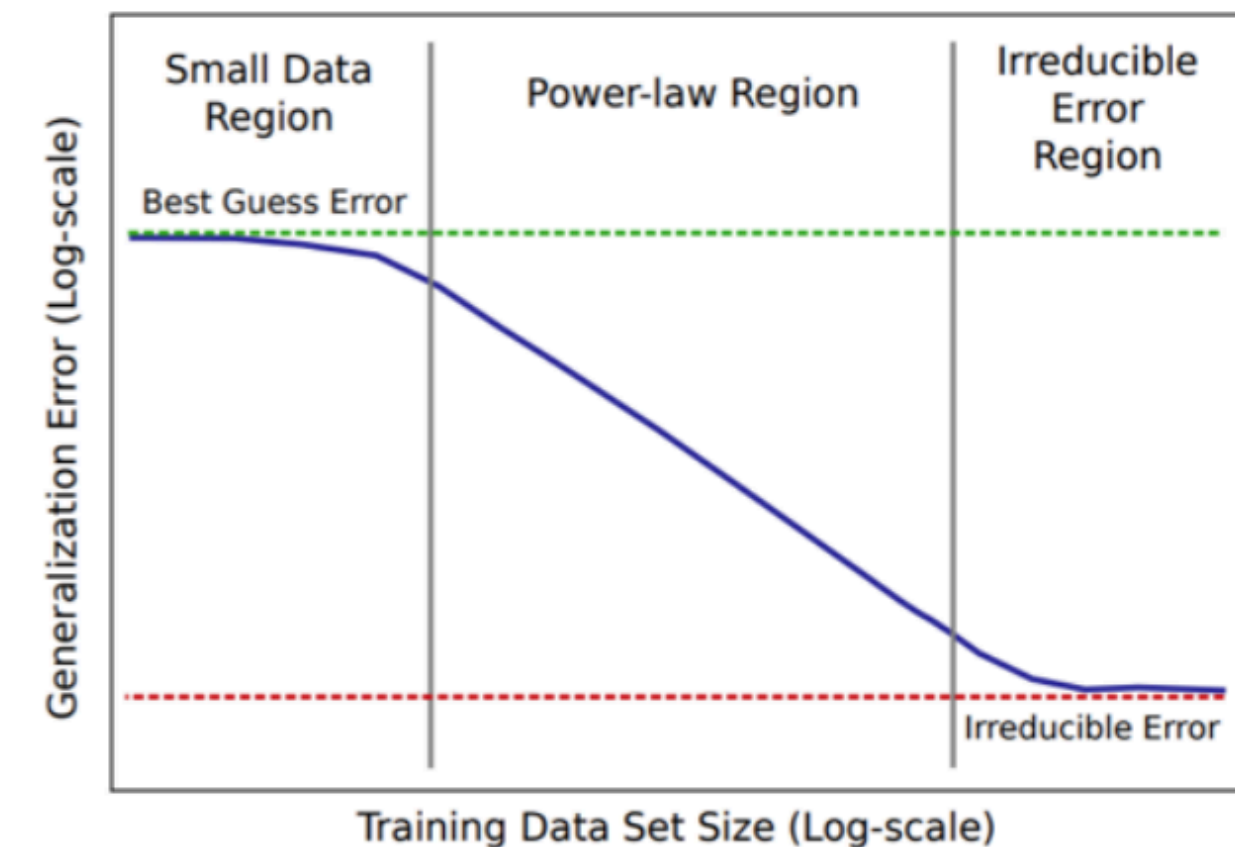
(For language modeling, from Kaplan+ 2020)

Foundations

Q: Why do scaling laws show up?

We know error should be monotone

But why is it a power law / linear in log-log?



A (?): Estimation error naturally decays polynomially.

But this answer may take a moment to understand. Let's work through an example.

Example: If our task is to estimate the mean of a dataset, what's the scaling law?

Example: Mean Estimation

Input: $x_1 \dots x_n \sim N(\mu, \sigma^2)$

Task: estimate the average as $\hat{\mu} = \frac{\sum_i x_i}{n}$

What's the error?

Example: Mean Estimation

Input: $x_1 \dots x_n \sim N(\mu, \sigma^2)$

Task: estimate the average as $\hat{\mu} = \frac{\sum_i x_i}{n}$

What's the error? By the Central Limit Theorem:

$$E[(\hat{\mu} - \mu)^2] = \frac{\sigma^2}{n}$$

This is a 'scaling law'

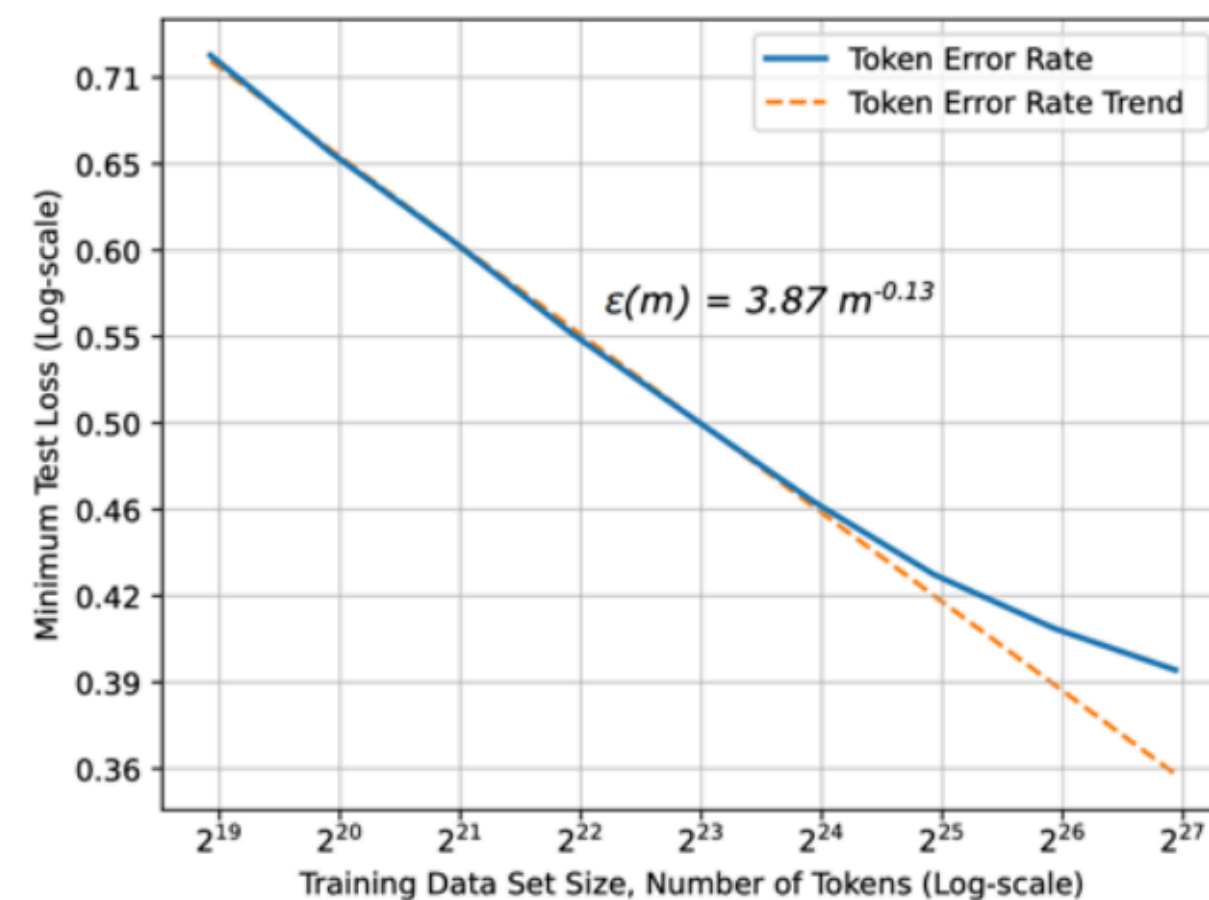
$$\log(\text{Error}) = -\log n + 2 \log \sigma$$

More generally, any polynomial rate $1/n^\alpha$ is a scaling law

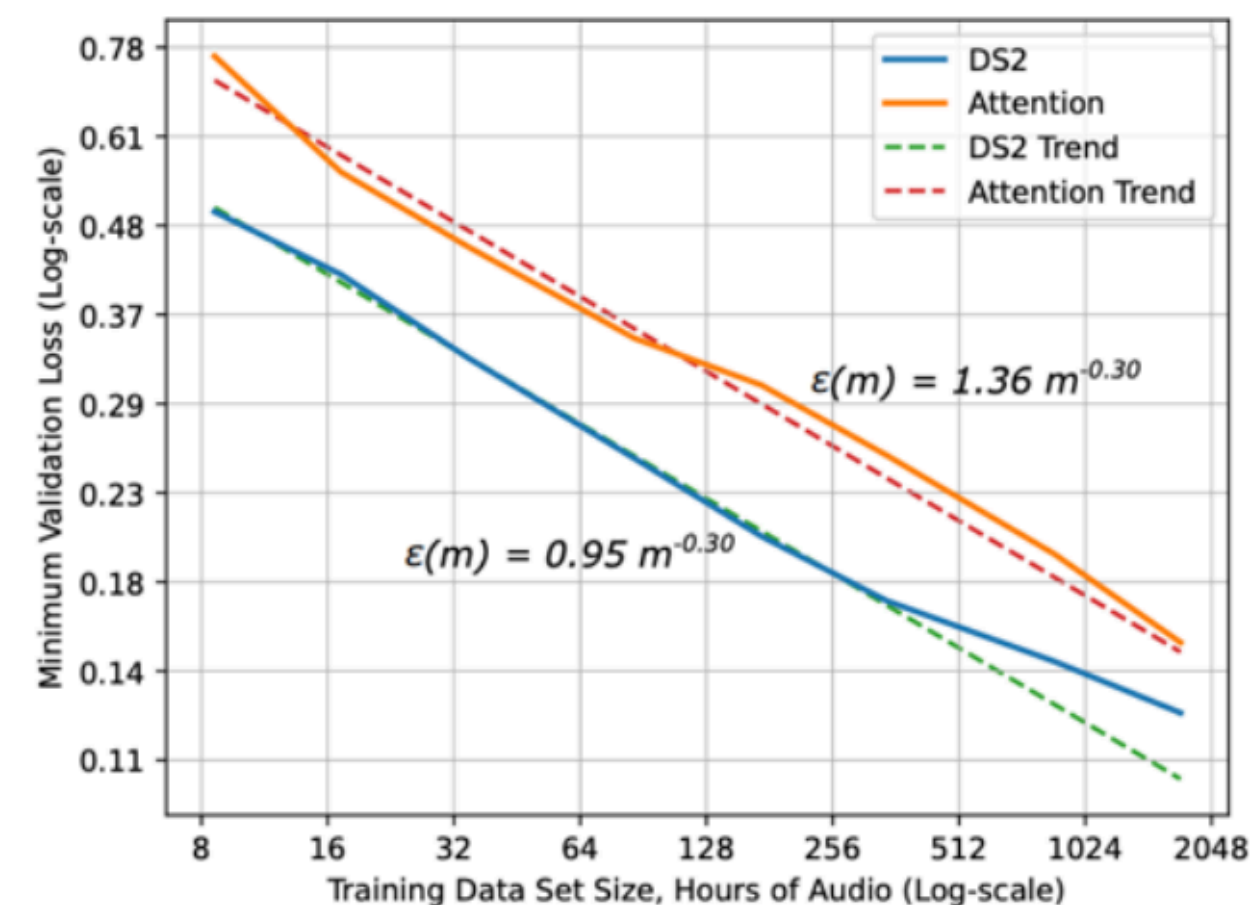
Scaling Laws in Practice

Fact: Similar arguments show most ‘classical’ models (regression, etc) have $\frac{1}{n}$ scaling

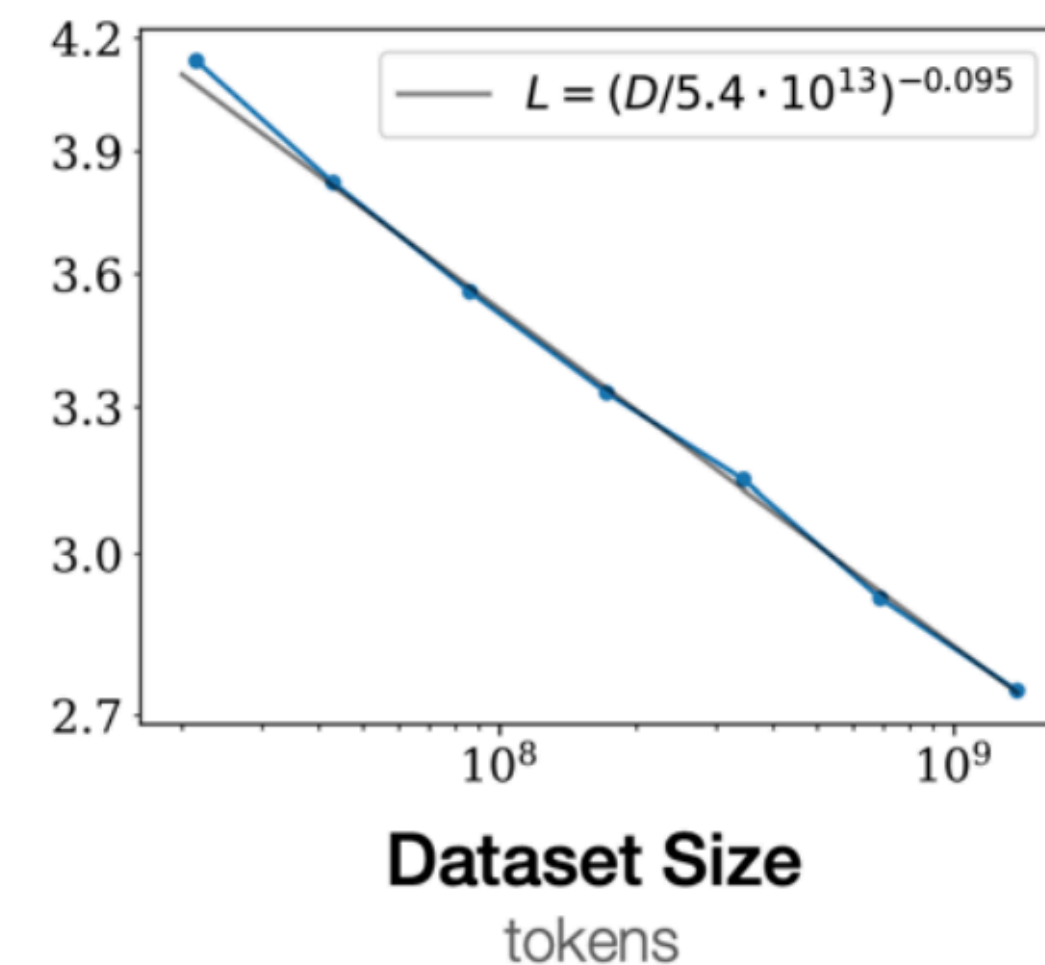
This means we should see $y = -x + C$
What do we find in neural scaling laws?



Machine translation



Speech



Language modeling

Very different from predictions.. Why might this be?

Function Approximation

Neural nets can approximate arbitrary functions. Lets turn that into an example.

Input: $x_1 \dots x_n$ uniform in 2D unit box. $y_i = f(x_i) + N(0,1)$

Task: estimate $f(x)$ by averaging over samples in the box

Approach: cut up the 2D space into boxes with length $n^{-\frac{1}{4}}$

What's our estimation error?

Informally, we have $\sqrt{n}$ boxes, each box gets $\sqrt{n}$ samples.

$$Error \approx \frac{1}{\sqrt{n}} + (\text{other smoothness terms})$$

(essentially the error of the mean estimate in the box)

Function Approximation

Neural nets can approximate arbitrary functions. Lets turn that into an example.

Input: $x_1 \dots x_n$ uniform in 2D unit box. $y_i = f(x_i) + N(0,1)$

Task: estimate $f(x)$ by averaging over samples in the box

Approach: cut up the 2D space into boxes with length $n^{-\frac{1}{4}}$

What's our estimation error?

Informally, we have $\sqrt{n}$ boxes, each box gets $\sqrt{n}$ samples.

$$Error \approx \frac{1}{\sqrt{n}} + (\text{other smoothness terms})$$

(essentially the error of the mean estimate in the box)

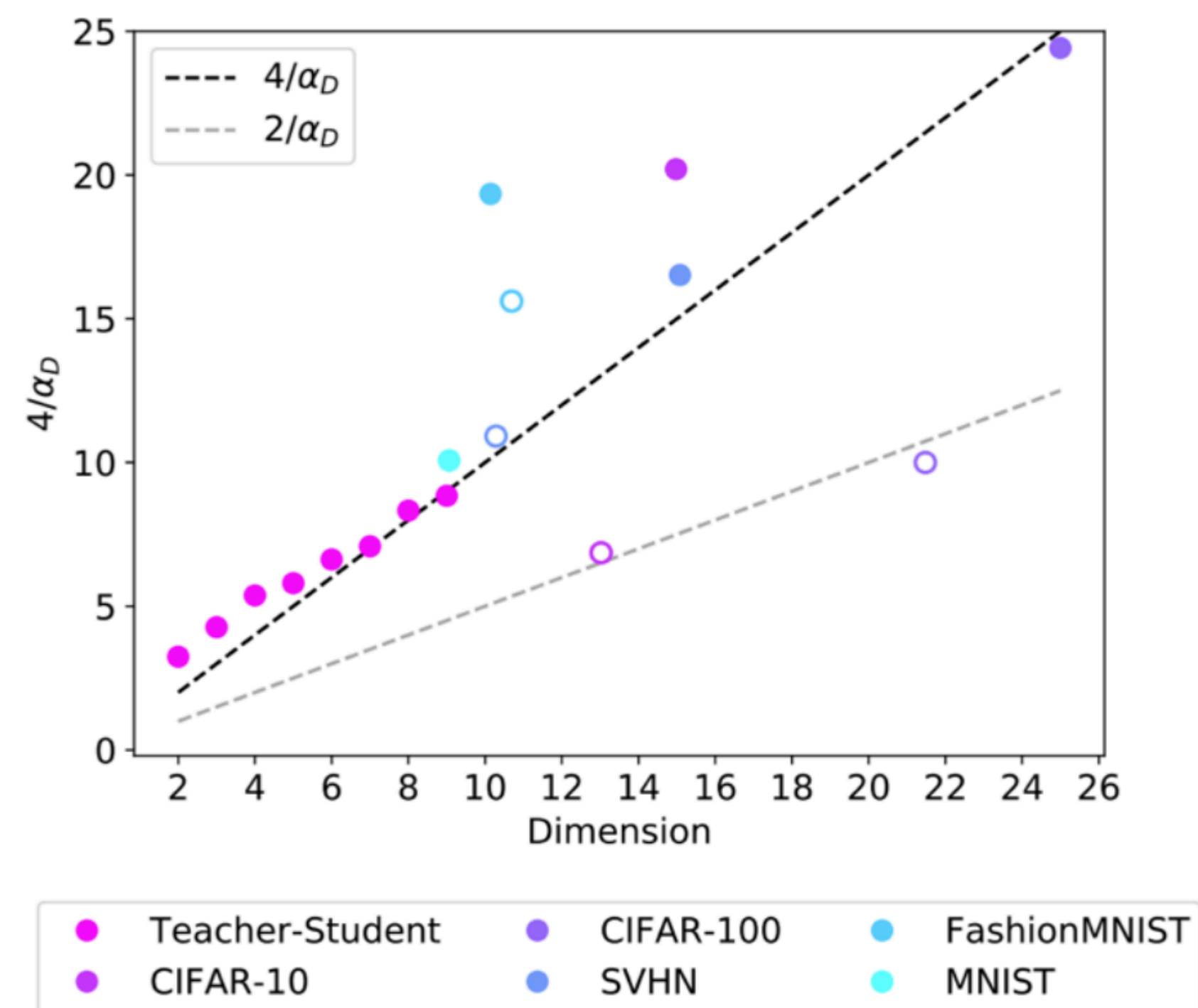
In d -dimensions, this becomes $Error = n^{-1/d}$ - **This means scaling is $y = -\frac{1}{d}x + C$**

Takeaway: flexible 'nonparametric' learning has dimension dependent scaling laws.

Intrinsic Dimensionality

Some have made the following argument (Bahri 2021)

1. Scaling laws arise due to polynomial rates of learning $\frac{1}{n^\alpha}$
2. Some argue the slope α is closely connected to the *intrinsic dimensionality* of the data.



Intrinsic Dimensionality

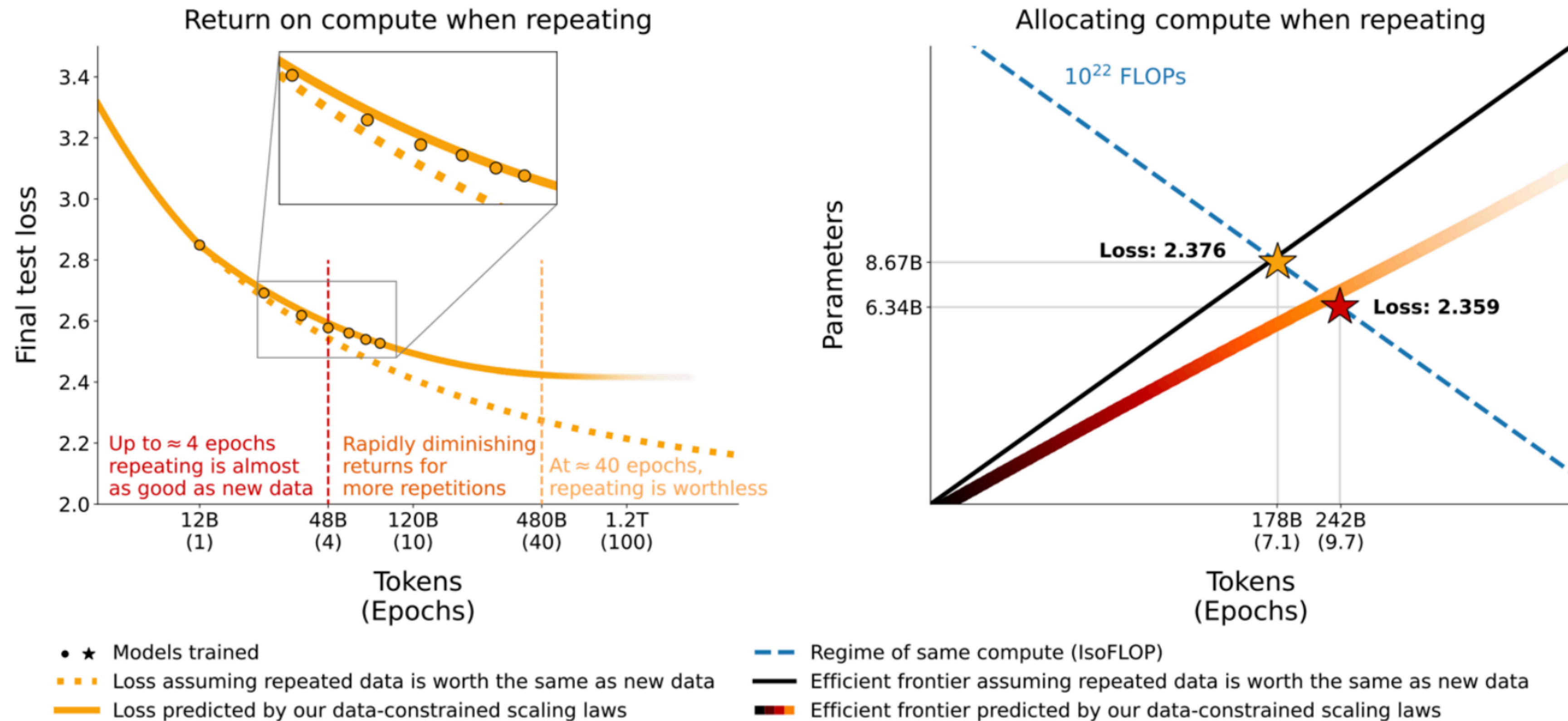
Data scaling thus far: how does dataset size relate to performance?

Related question: how does dataset *composition* affect performance

- ❖ Picking optimal data mixture using small scale models
(we'll revisit this later)
- ❖ Deciding whether to repeat data or not
- ❖ Combining the two and balancing quality with repetition rate

Repeating Data

In practice, we have finite data – how does repeating examples affect scaling?



Muenninghoff et al. (2023)

$$D' = U_D + U_D R_D^* (1 - e^{\frac{-R_D}{R_D^*}}) .$$

D' = Effective data
 U_d = Unique tokens
 R_d^* = Constant
 R_d = Repetition

Data Scaling Laws

- ❖ Remarkably linear relationship between log-data size and log-error
- ❖ Holds across domains and models
- ❖ Theory understanding: similar to generalization bounds: mean estimation example
- ❖ Applications: data collection / curation

Intro to Pre-training and Prompting

Data Scaling Laws

Model Scaling Laws

Chinchilla: Data x Model

Putting it together

Model Scaling Laws

Slide credit: Tatsu Hashimoto

Model Scaling Laws

Our motivation: how can we efficiently design huge LMs?

- LSTMs vs Transformers
- Adam vs SGD

How should we allocate our limited resources?

- Train models longer vs train bigger models?
- Collect more data vs get more GPUs?

Scaling laws provide a simple procedure to answer these.

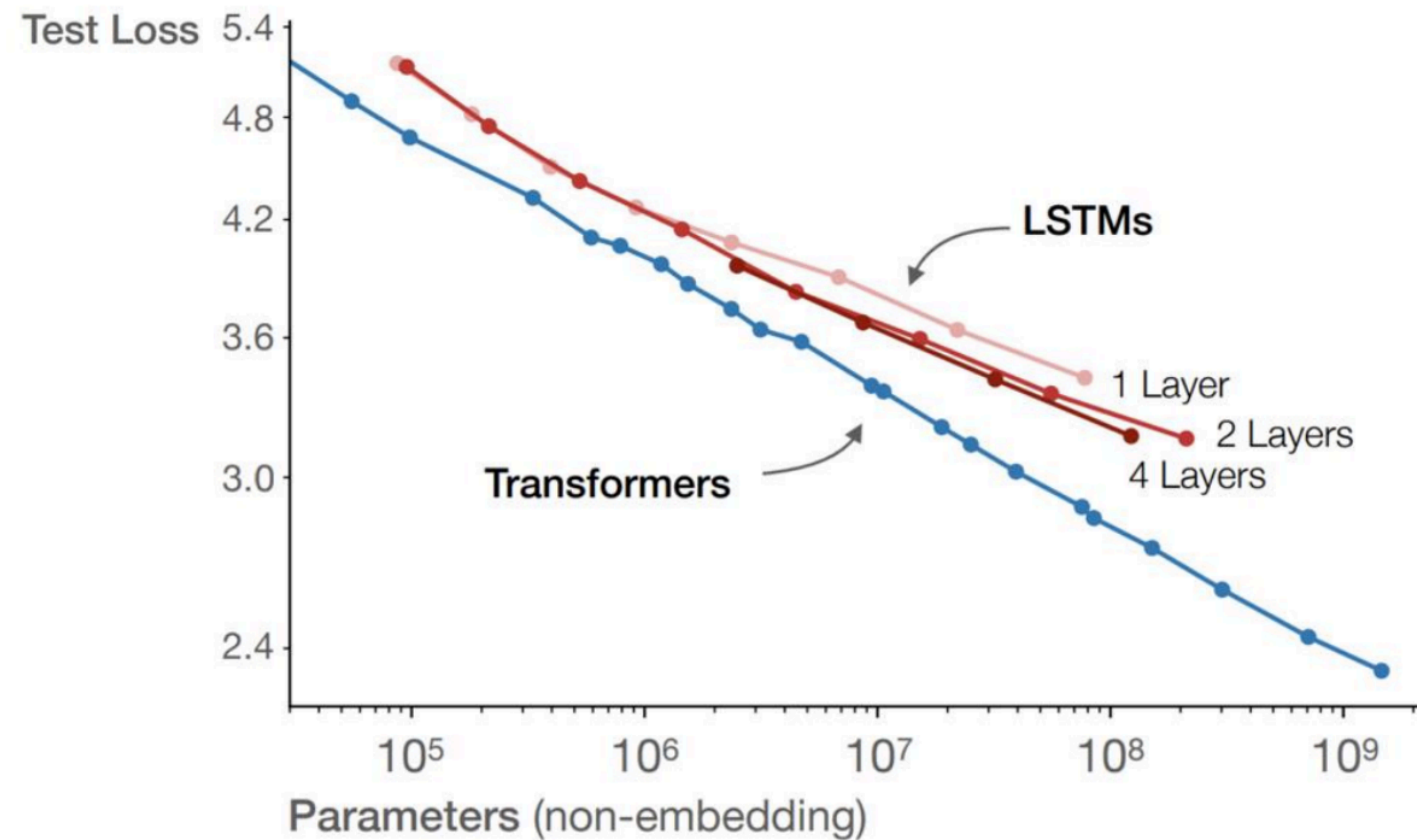
Consider: 1. Architecture; 2. Optimizer; 3. Aspect ratio/depth; 4. Batch size

Architecture

Q: Are transformers better than LSTMs?

Brute force way: spend tens of millions to train a LSTM GPT-3

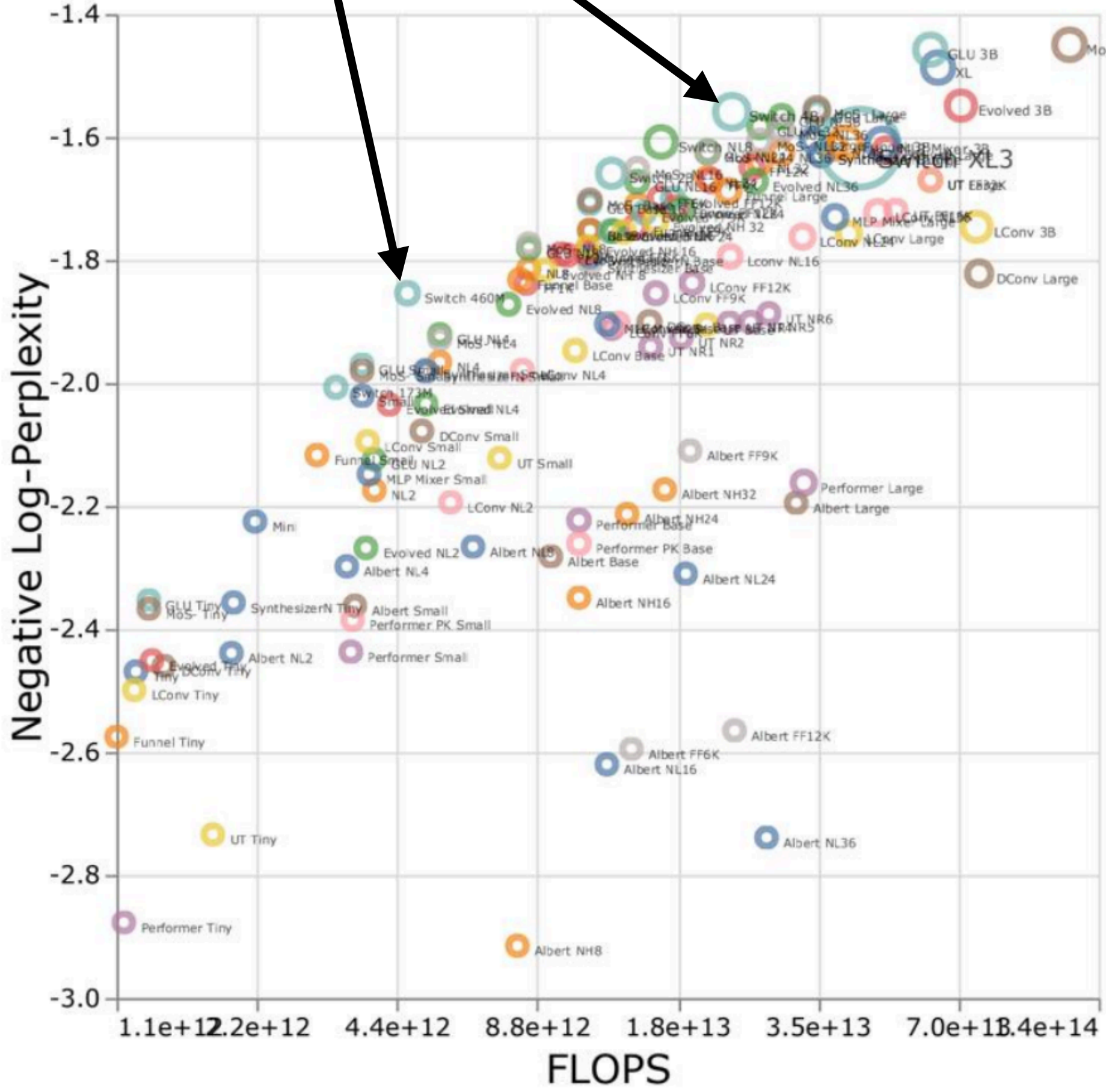
Scaling law way:



[Kaplan+ 2021]

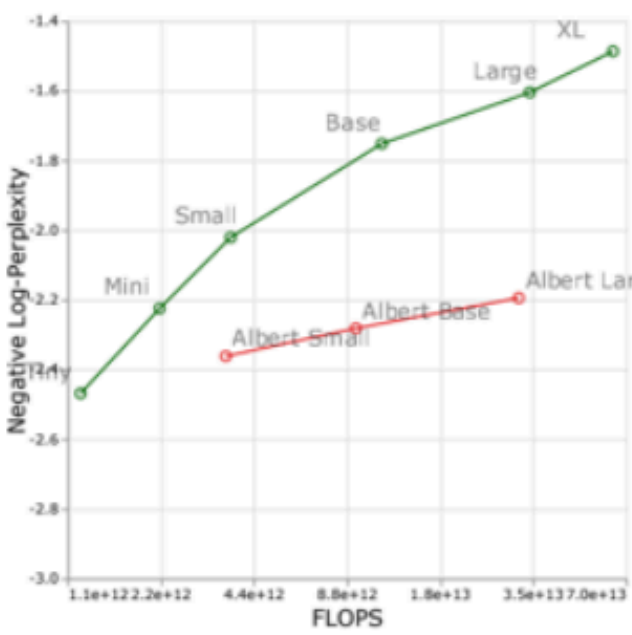
Switch Transformer Architecture

1. Many architectures

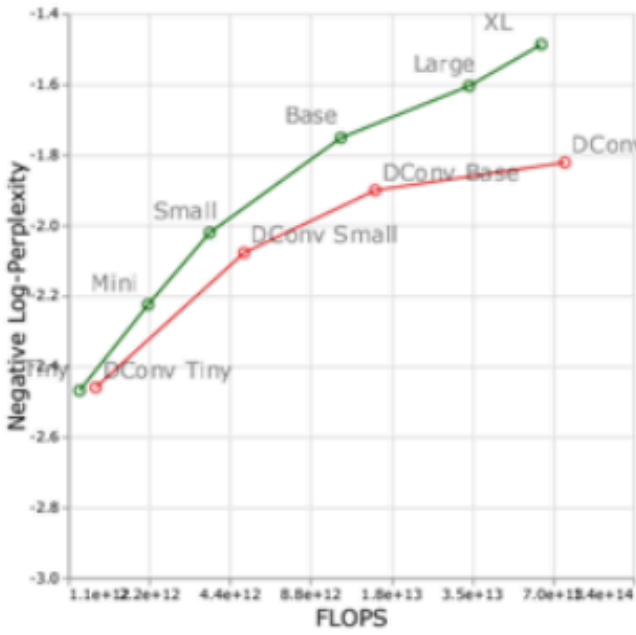


size = num params

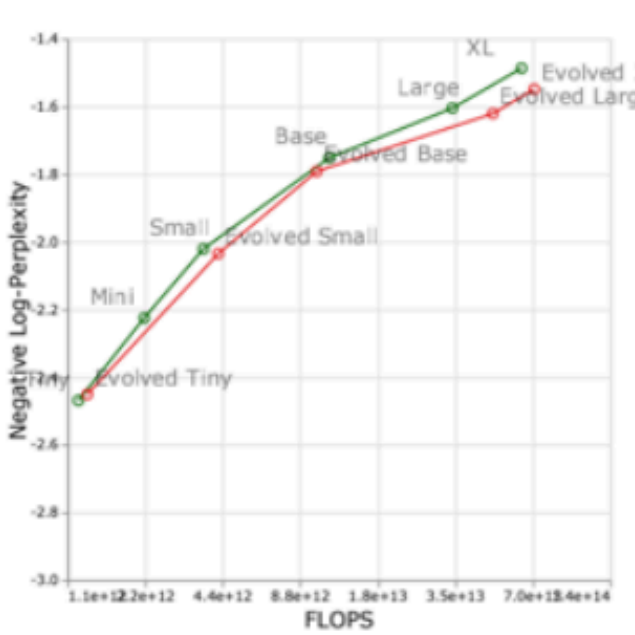
Tay et al. (2022)



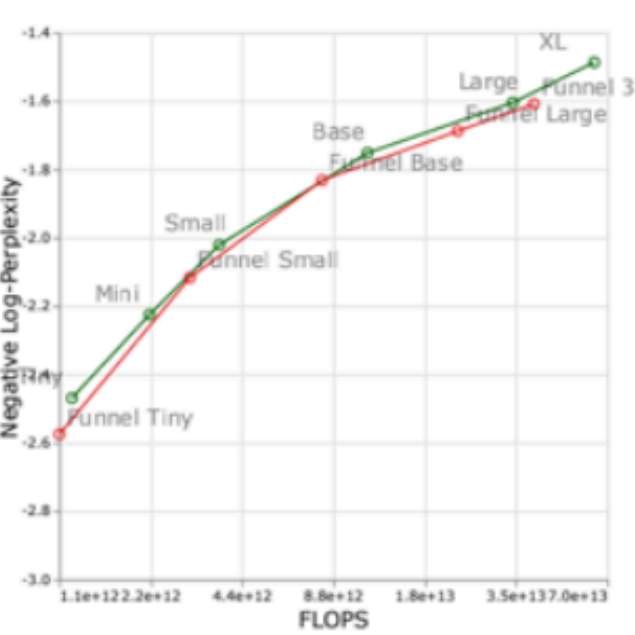
(a) ALBERT



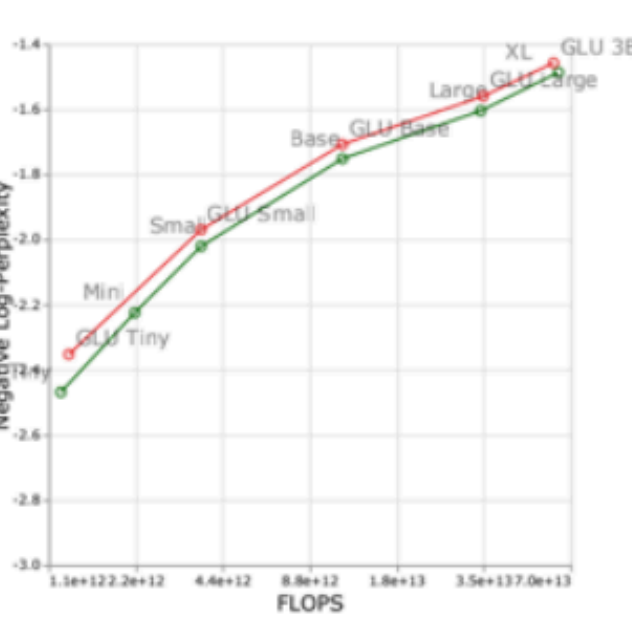
(b) DConv



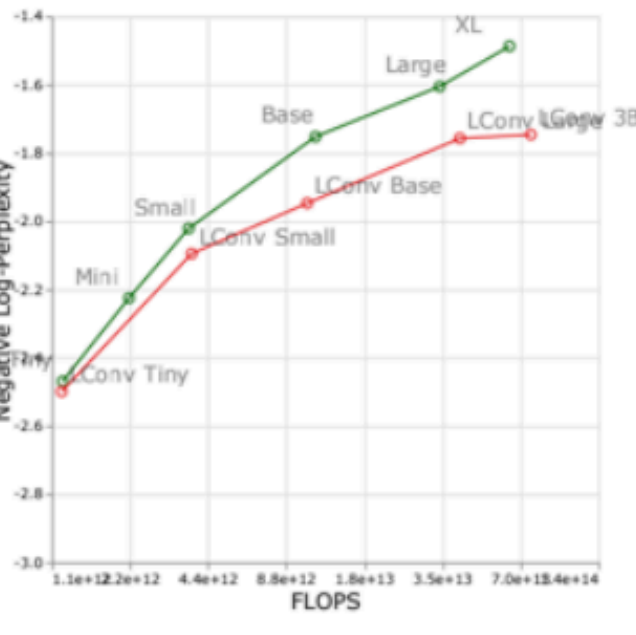
(c) Evolved



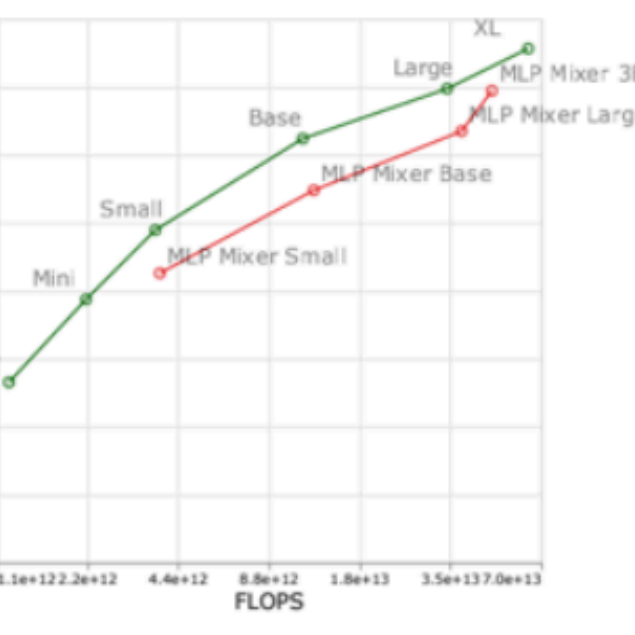
(d) Funnel



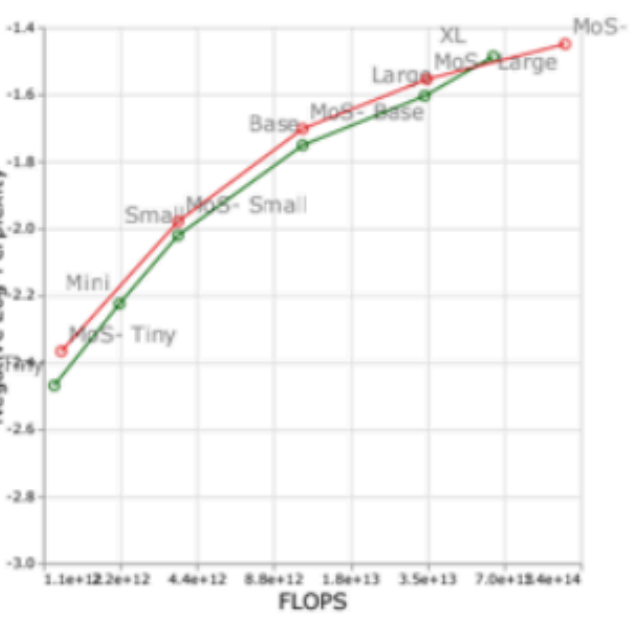
(e) Transformer-GLU



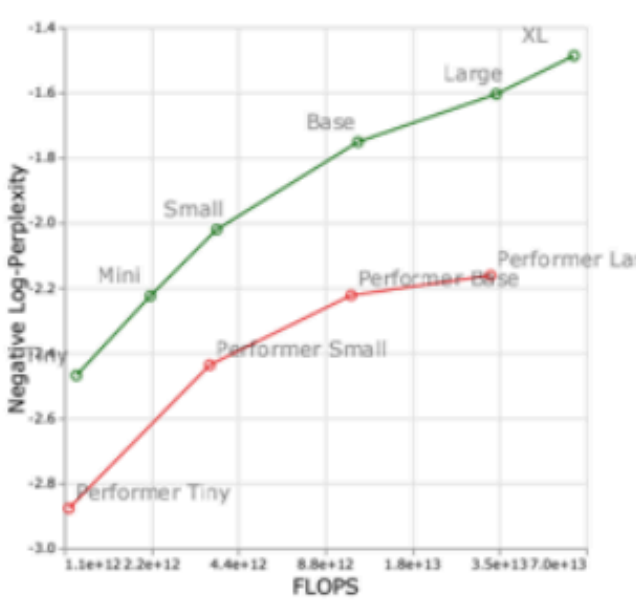
(f) LConv



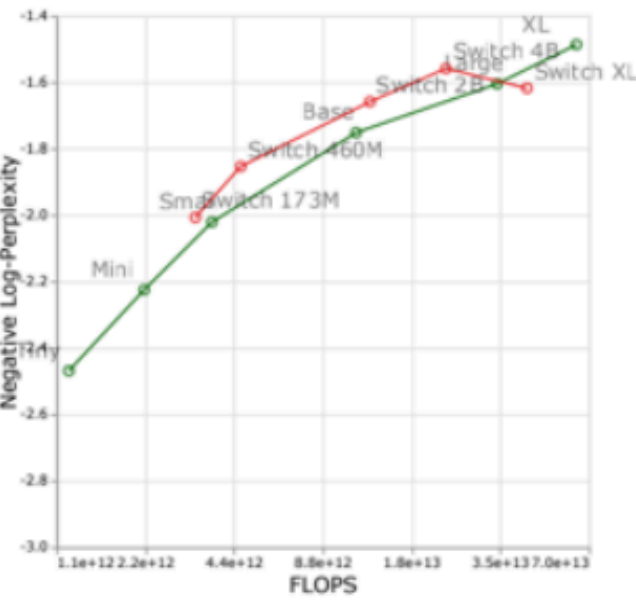
(g) MLP Mixer



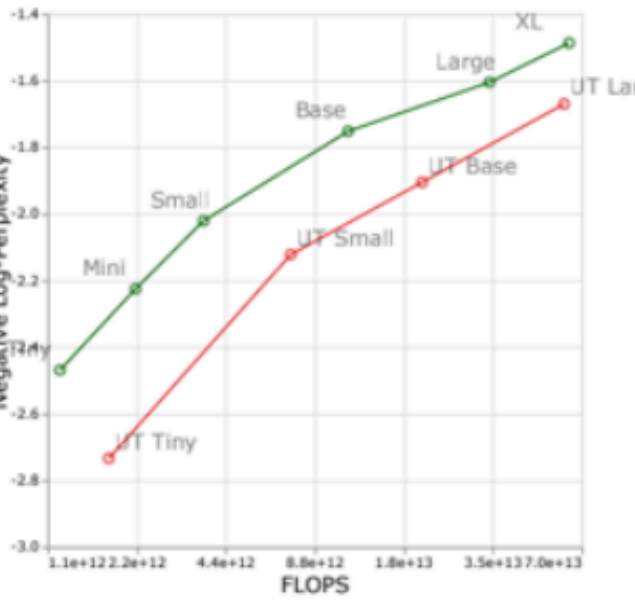
(h) MoS Transformer



(i) Performer



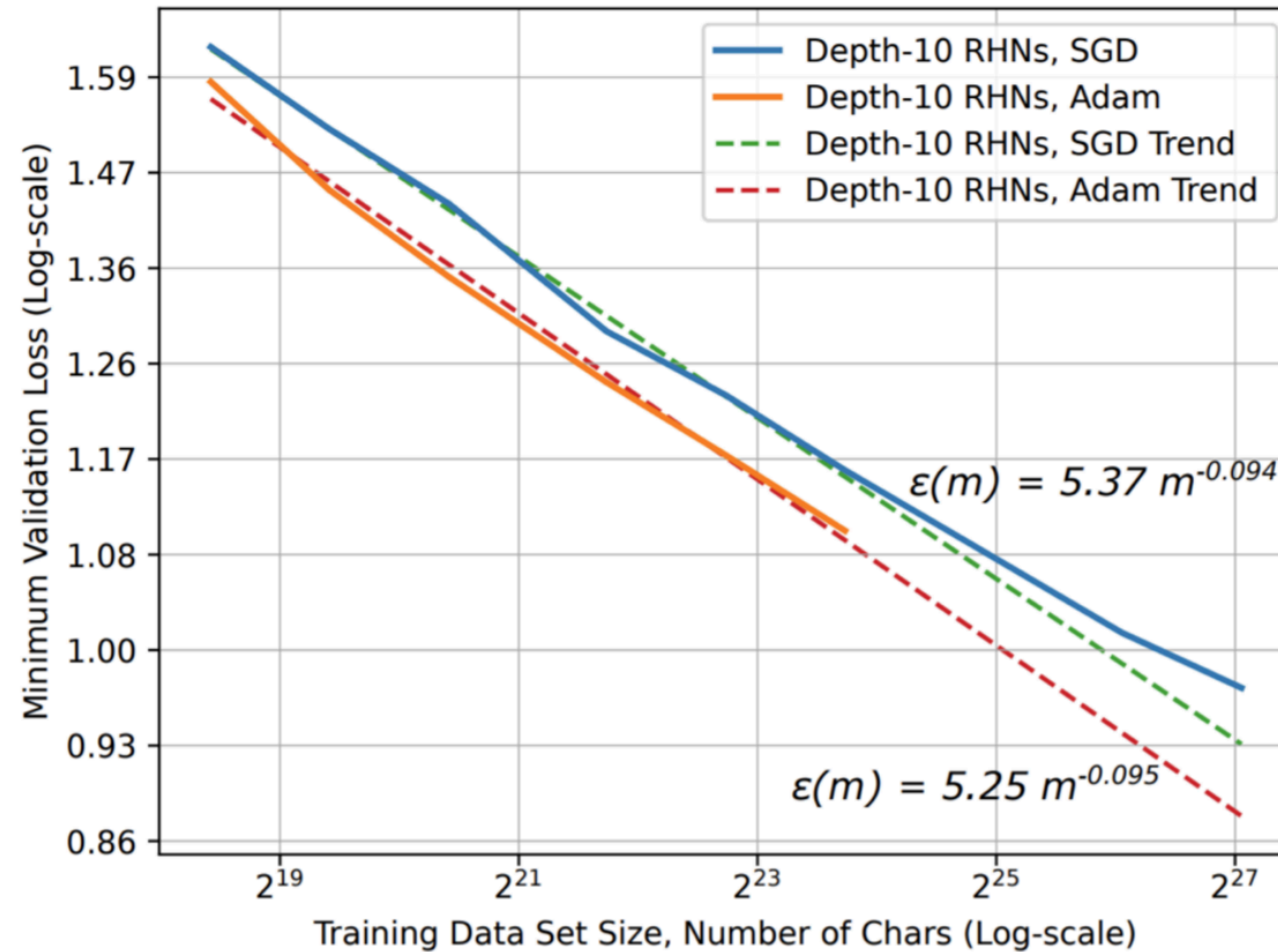
(j) Switch Transformer



(k) Universal Transformer

Optimizer

What about ADAM vs SGD?

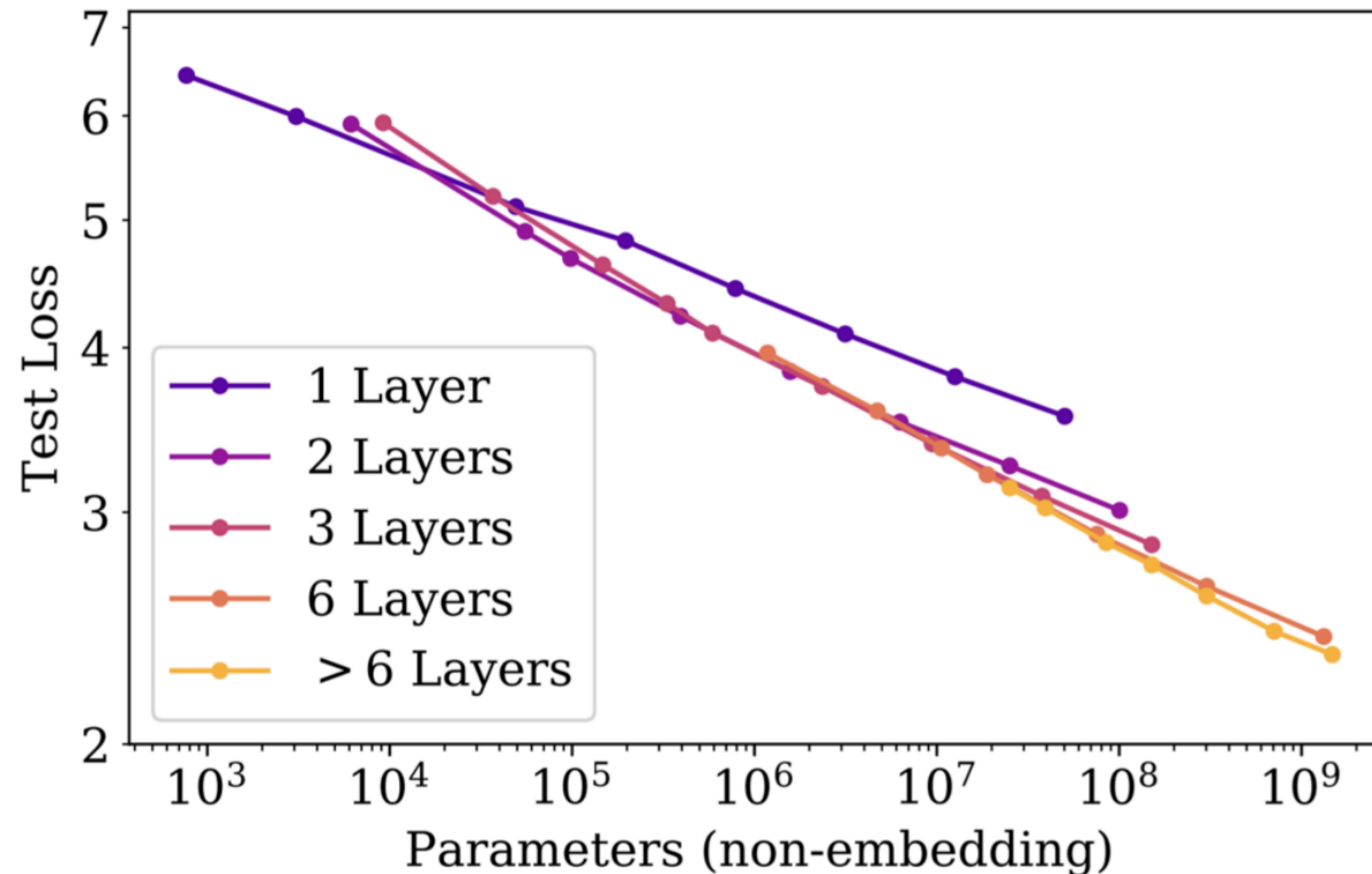


[Hestness+ 2017]

(Note, this is in 2017, so pre-transformers. RHN is recurrent highway nets)

Depth/Width

Does depth or width make a huge difference?

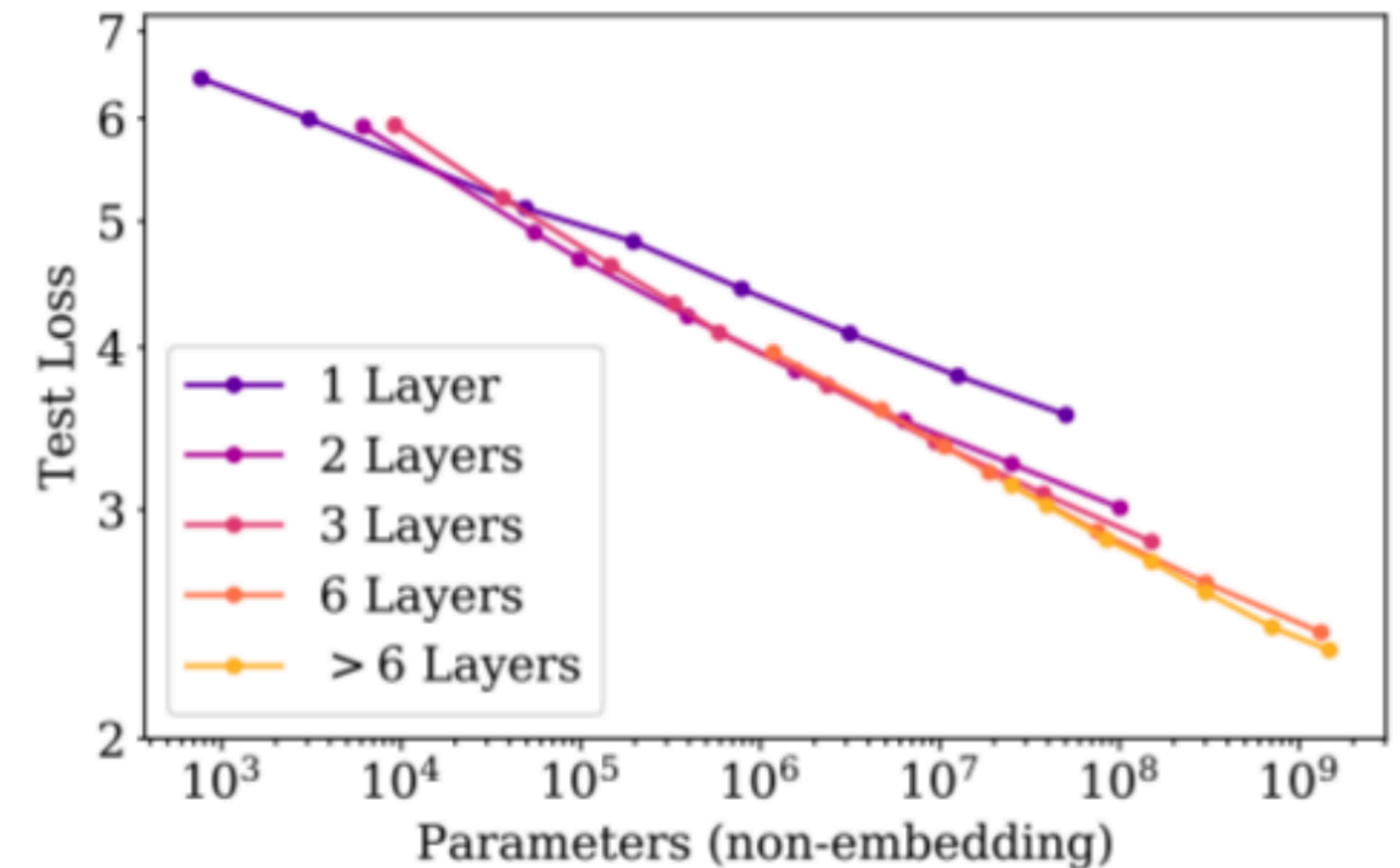
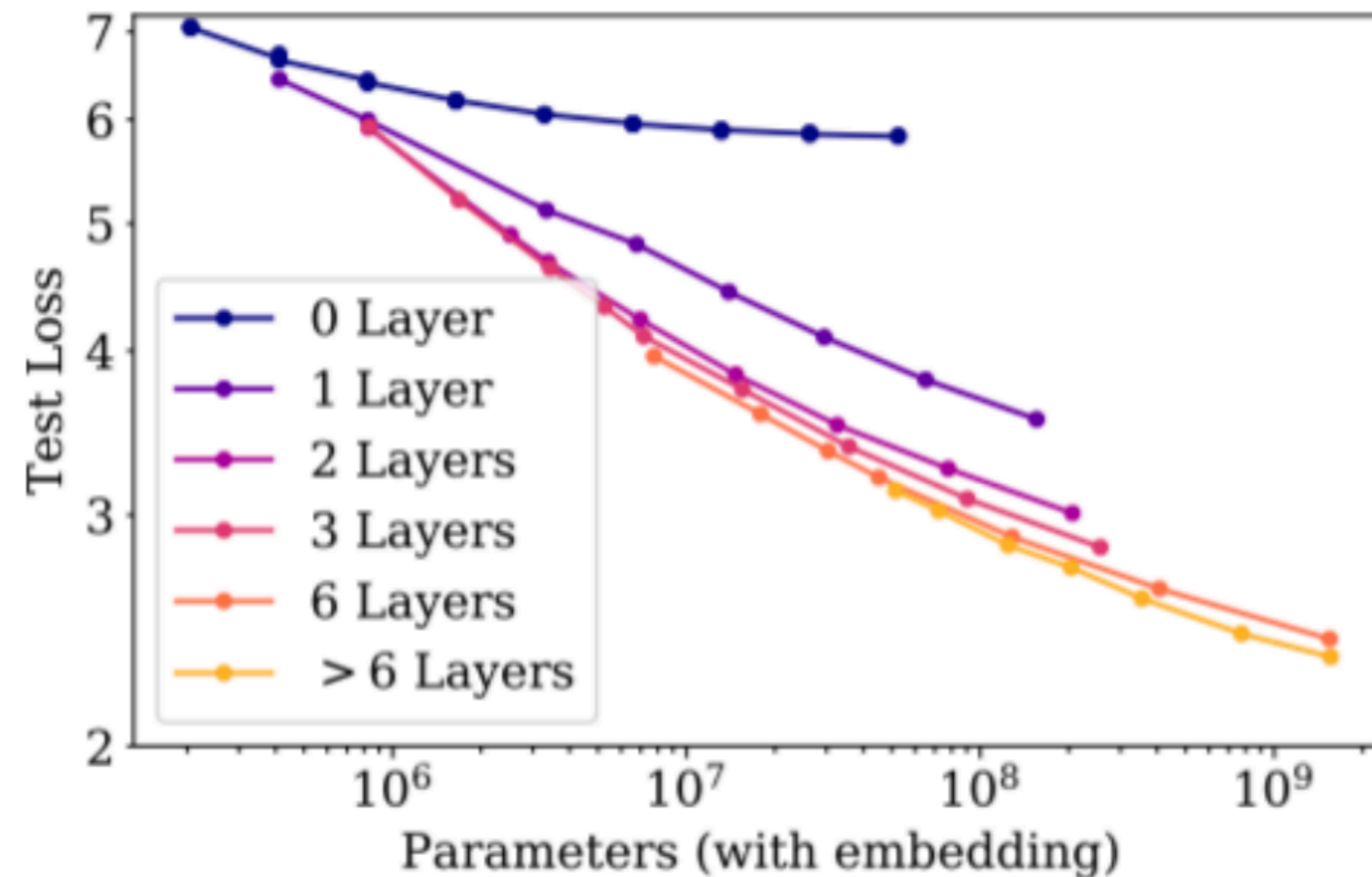


Kaplan et al. (2020)

- 1 vs 2 layers makes a huge difference.
- More layers have diminishing returns below 10^7 params

Depth/Width

We've been thinking about 'parameters' but not all parameters are equal



Embedding layer parameters don't behave the same!

Kaplan et al. (2020)

Related: recent papers on scaling laws for mixtures of experts.

Depth/Width

Do hyperparameters like the aspect ratio depend on scale?

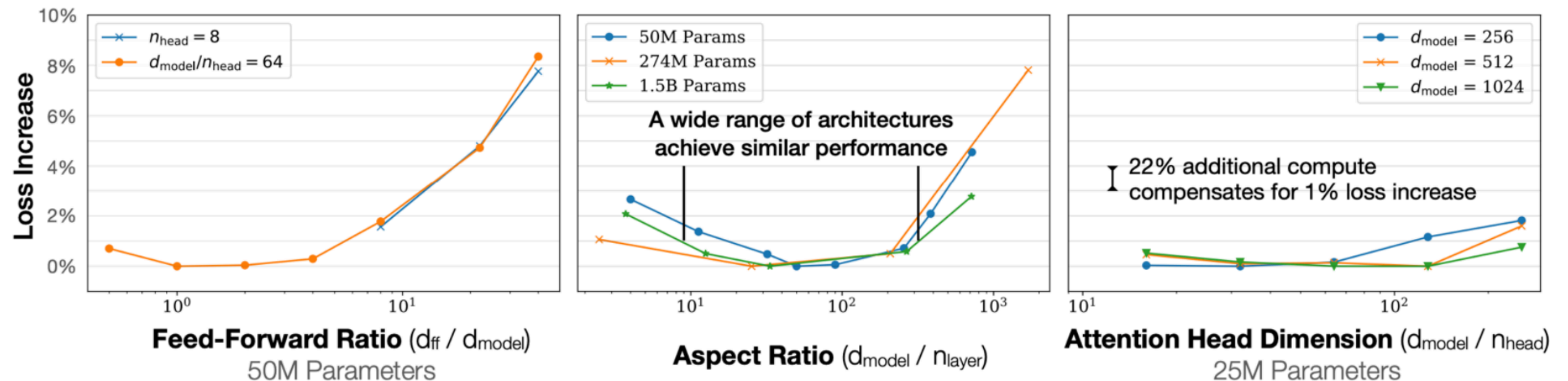
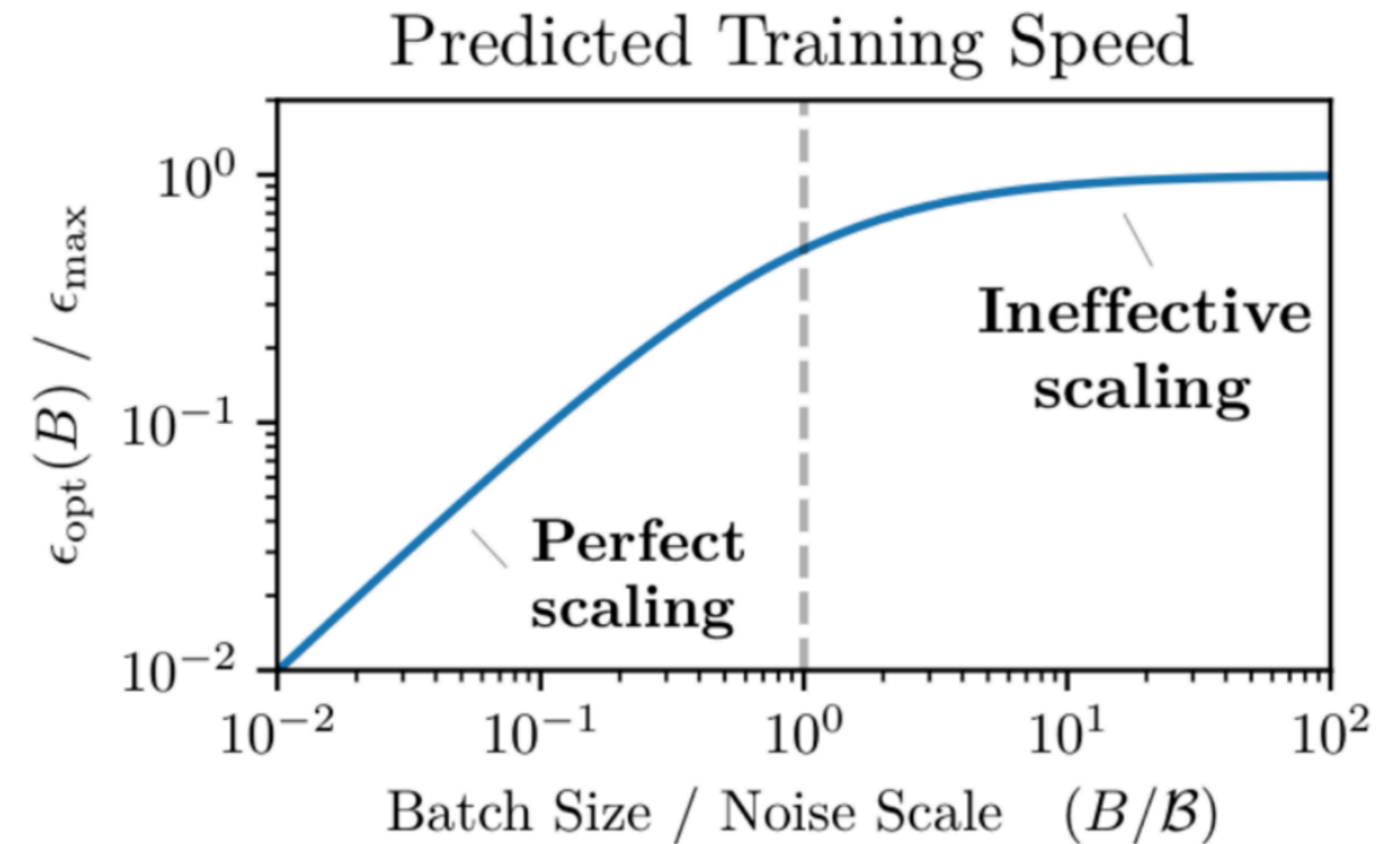
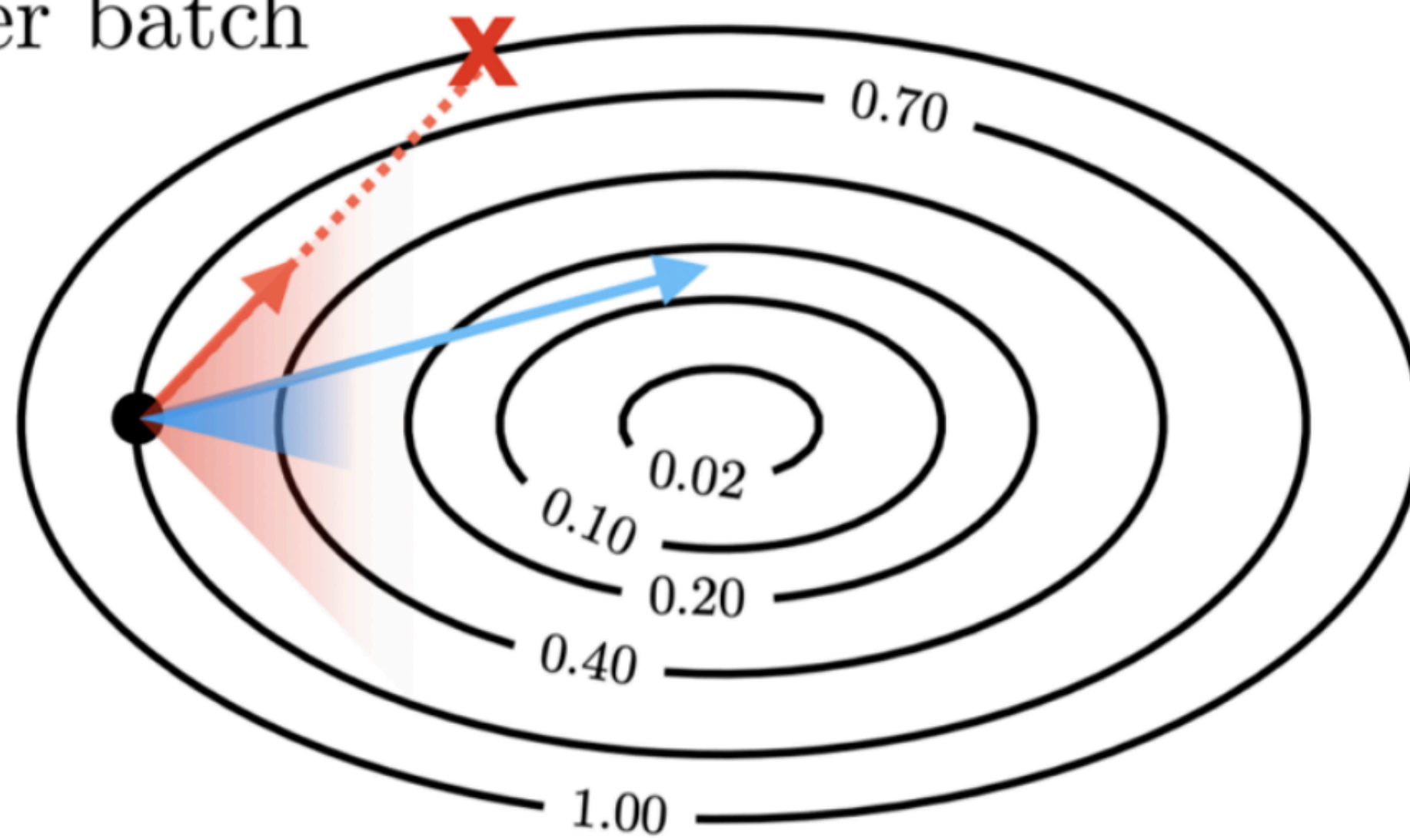


Figure 5 Performance depends very mildly on model shape when the total number of non-embedding parameters N is held fixed. The loss varies only a few percent over a wide range of shapes. Small differences in parameter counts are compensated for by using the fit to $L(N)$ as a baseline. Aspect ratio in particular can vary by a factor of 40 while only slightly impacting performance; an $(n_{layer}, d_{model}) = (6, 4288)$ reaches a loss within 3% of the $(48, 1600)$ model used in [RWC⁺19].

Kaplan et al. (2020)

Batch Size

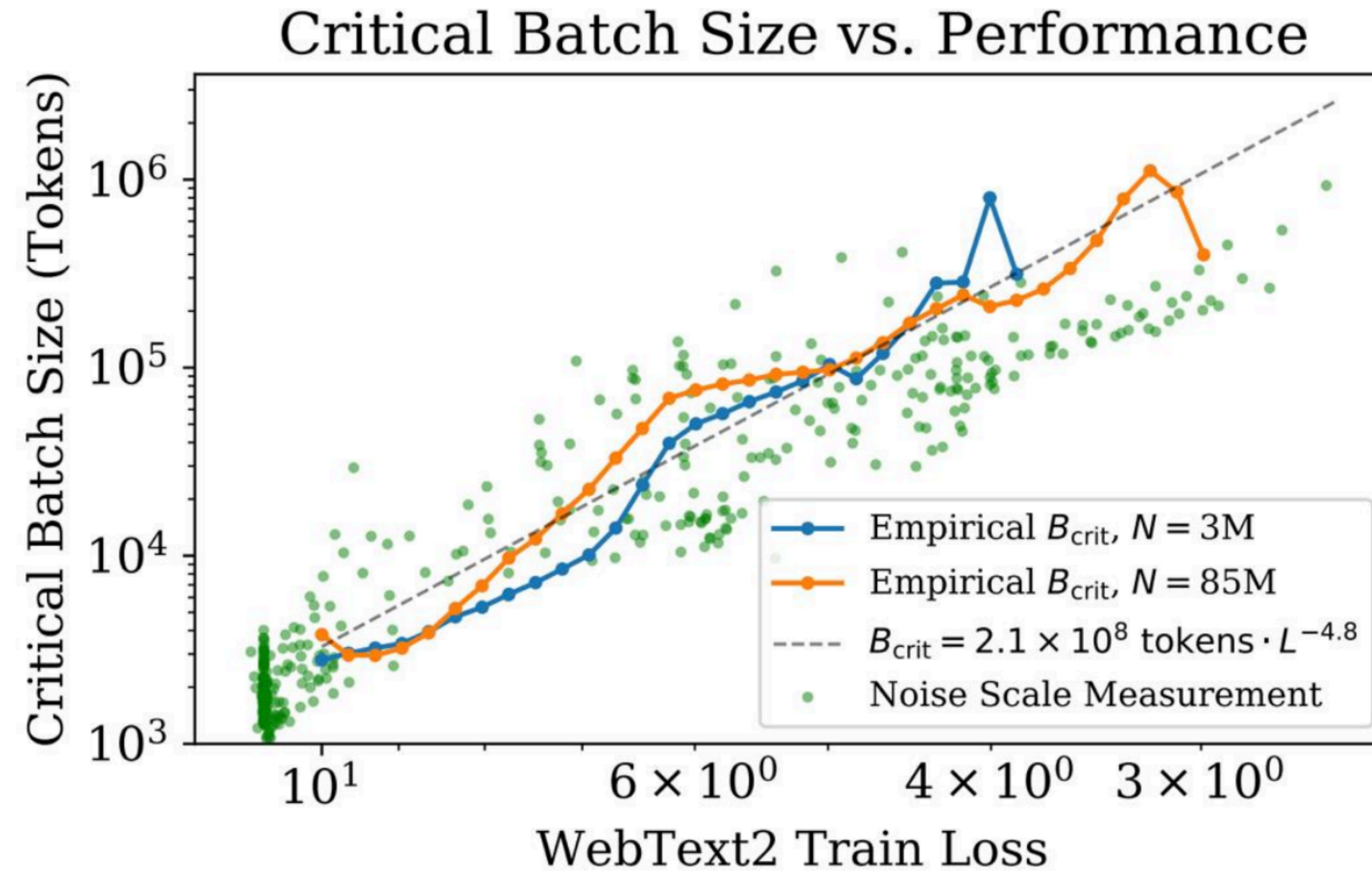
- Smaller batch
- Larger batch



Batch size – known to have strong diminishing returns past a certain point.

Critical batch = min number of examples for target loss / min number of steps for target loss

Batch Size



The smaller the loss target,
The bigger the batch

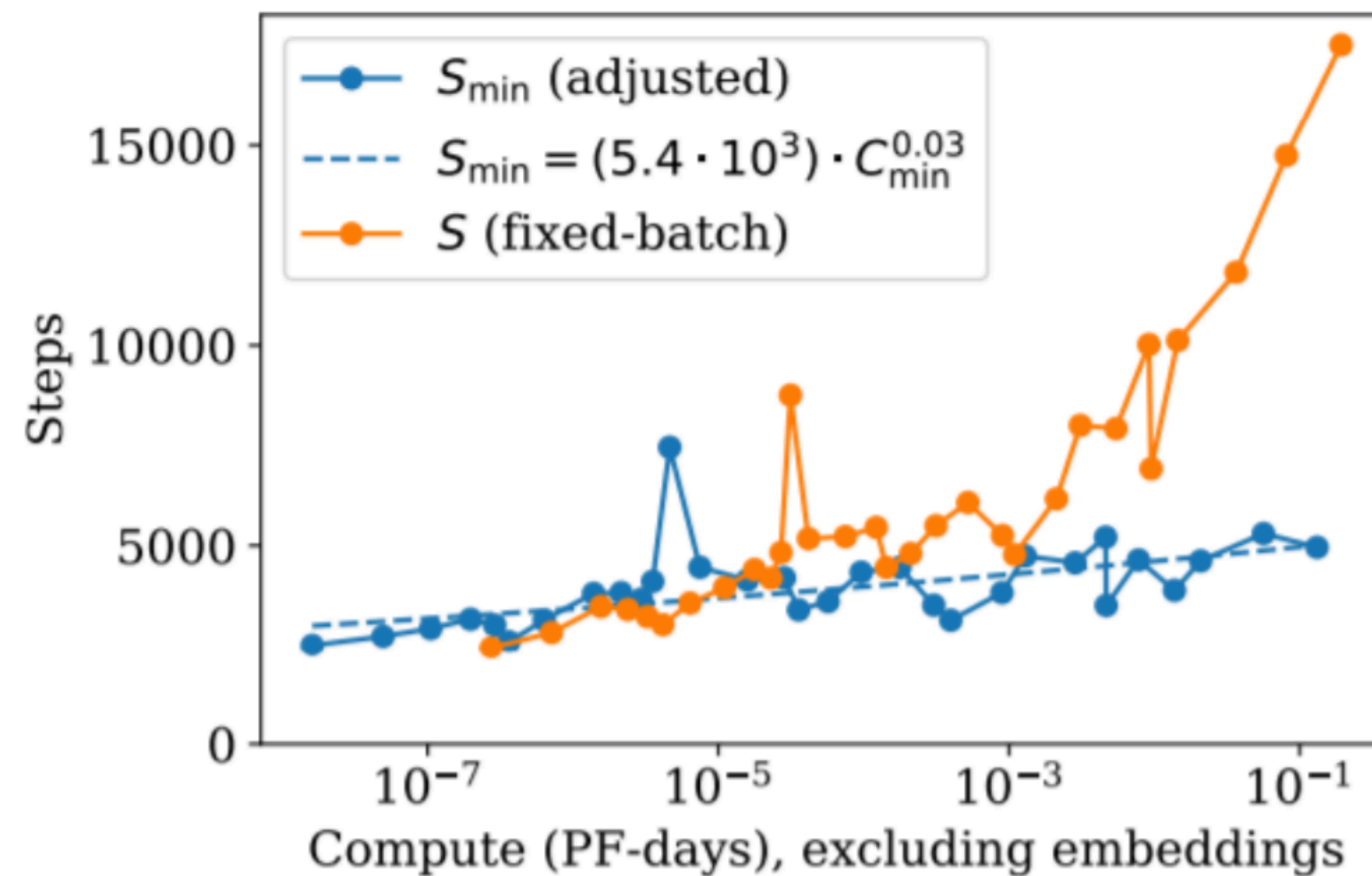
$$C_{\min}(C) \equiv \frac{C}{1 + B/B_{\text{crit}}(L)} \quad (\text{minimum compute, at } B \ll B_{\text{crit}})$$

Kaplan et al. (2020)

Batch Size

Q: as we increase both compute and model size, how should we scale training?

- Huge batches, same number of steps
- Fixed batches, more steps



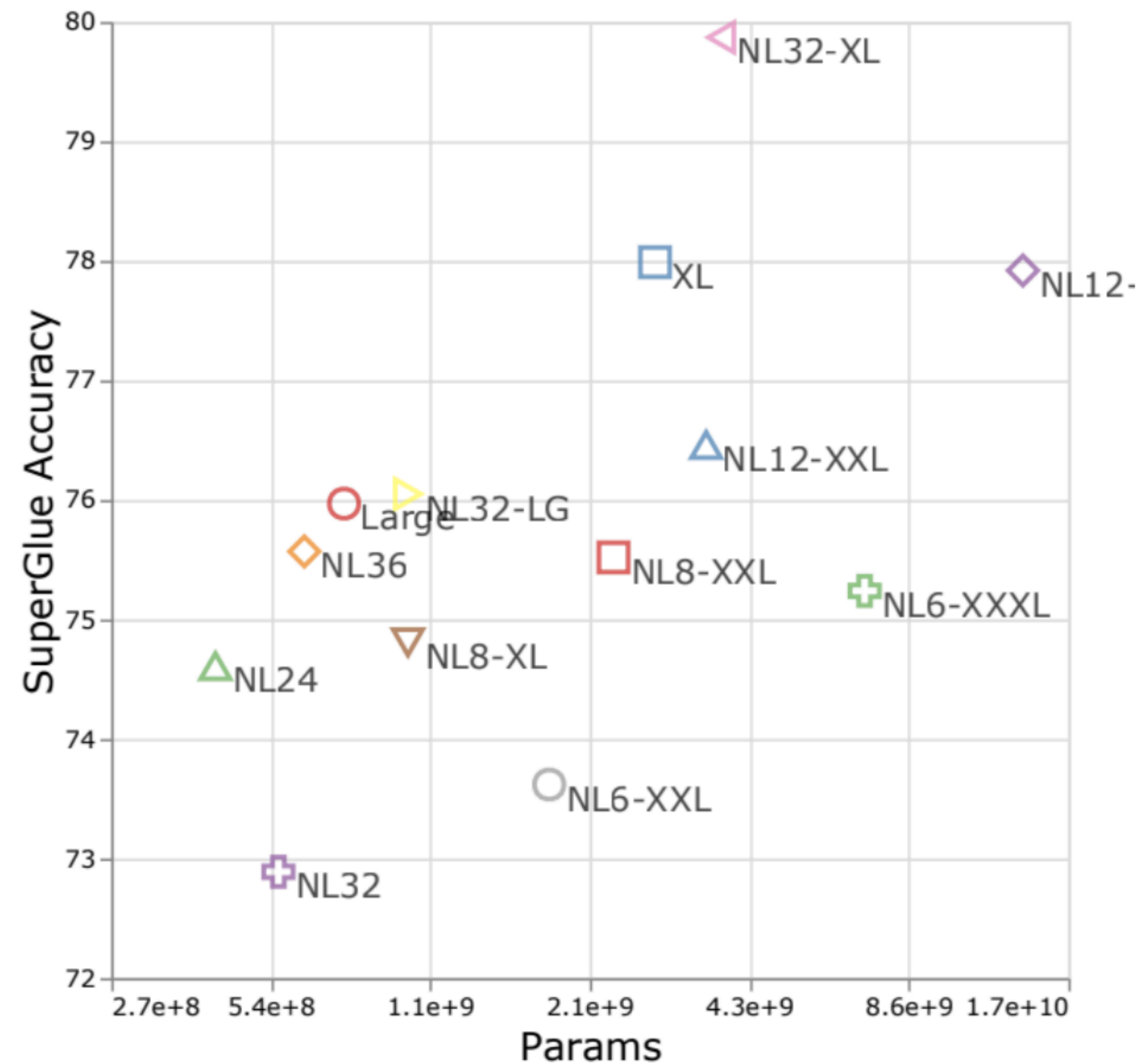
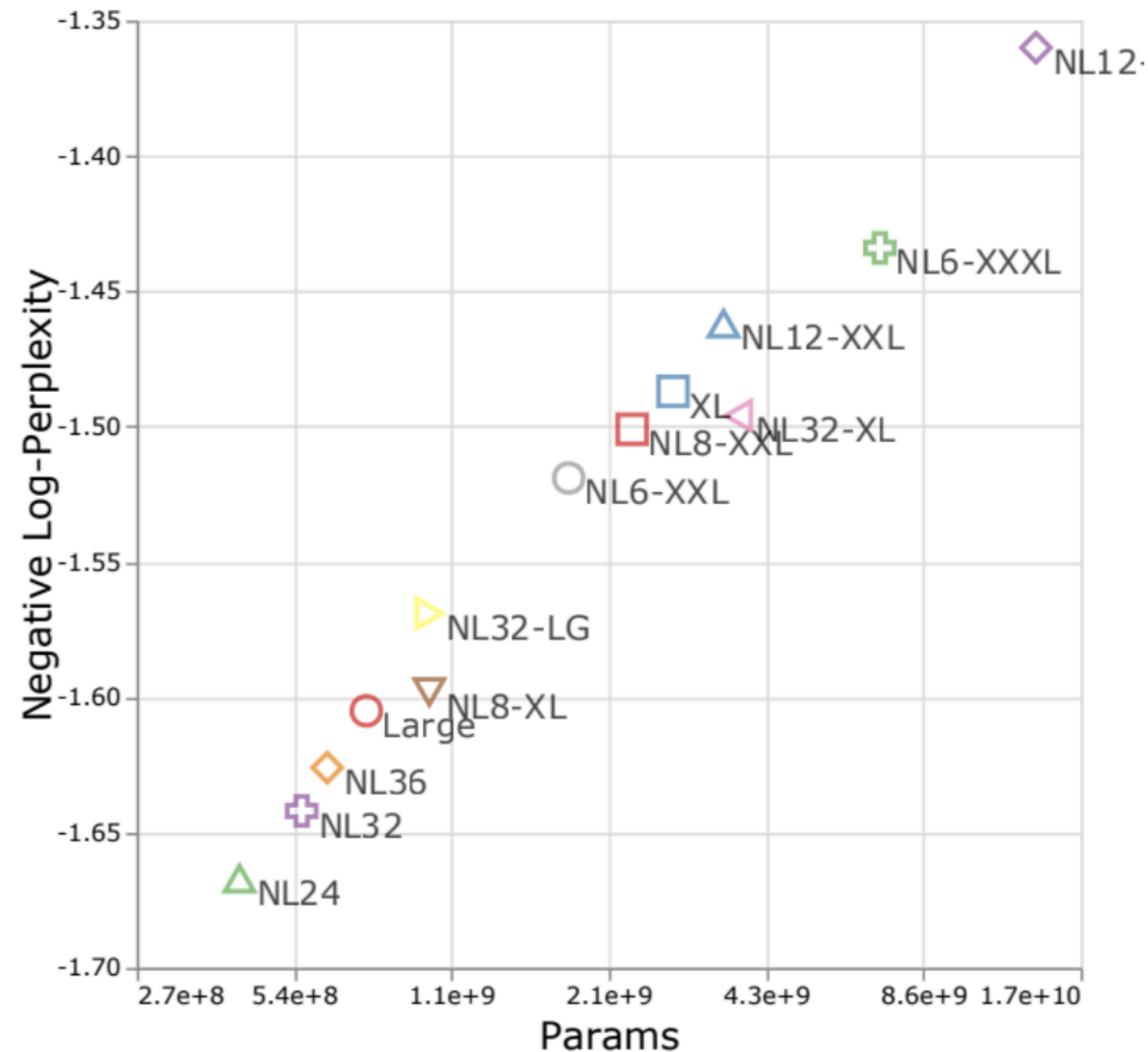
Kaplan et al. (2020)

Good news for data parallel processing (?)

Scaling: Downstream Eval

Thus far: scaling is predictable and depends mainly on parameters

Catch: downstream scaling can often be much less predictable



Scaling: Downstream Eval

The effect of hyperparameters on big LMs can be predicted *before* training!

- Optimizer choice
 - Model depth
- Architecture choice

The scaling law based design procedure.

1. Train a few smaller models
2. Establish a scaling law (e.g. ADAM vs SGD scaling law)
3. Select optimal hyperparam based on the scaling law prediction.

Intro to Pre-training and Prompting

Data Scaling Laws

Model Scaling Laws

Chinchilla: Data x Model

Putting it together

Chinchilla: Data x Model

Slide credit: Tatsu Hashimoto

Model x Data Scaling

Joint data-model scaling laws describe how the two relate

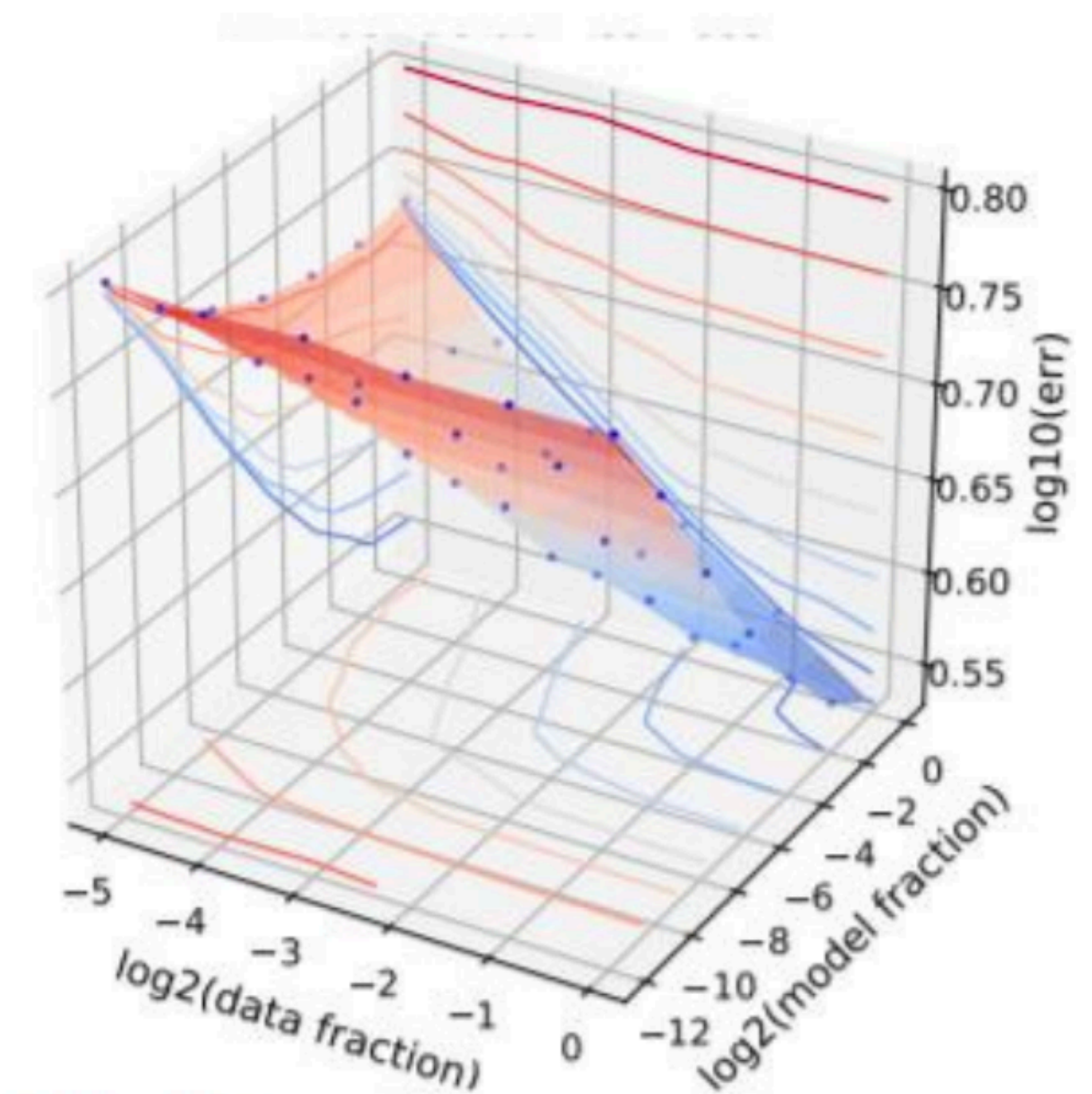
From Rosenfeld+ 2020,

$$Error = n^{-\alpha} + m^{-\beta} + C$$

From Kaplan+ 2020

$$Error = [m^{-\alpha} + n^{-1}]^{\beta}$$

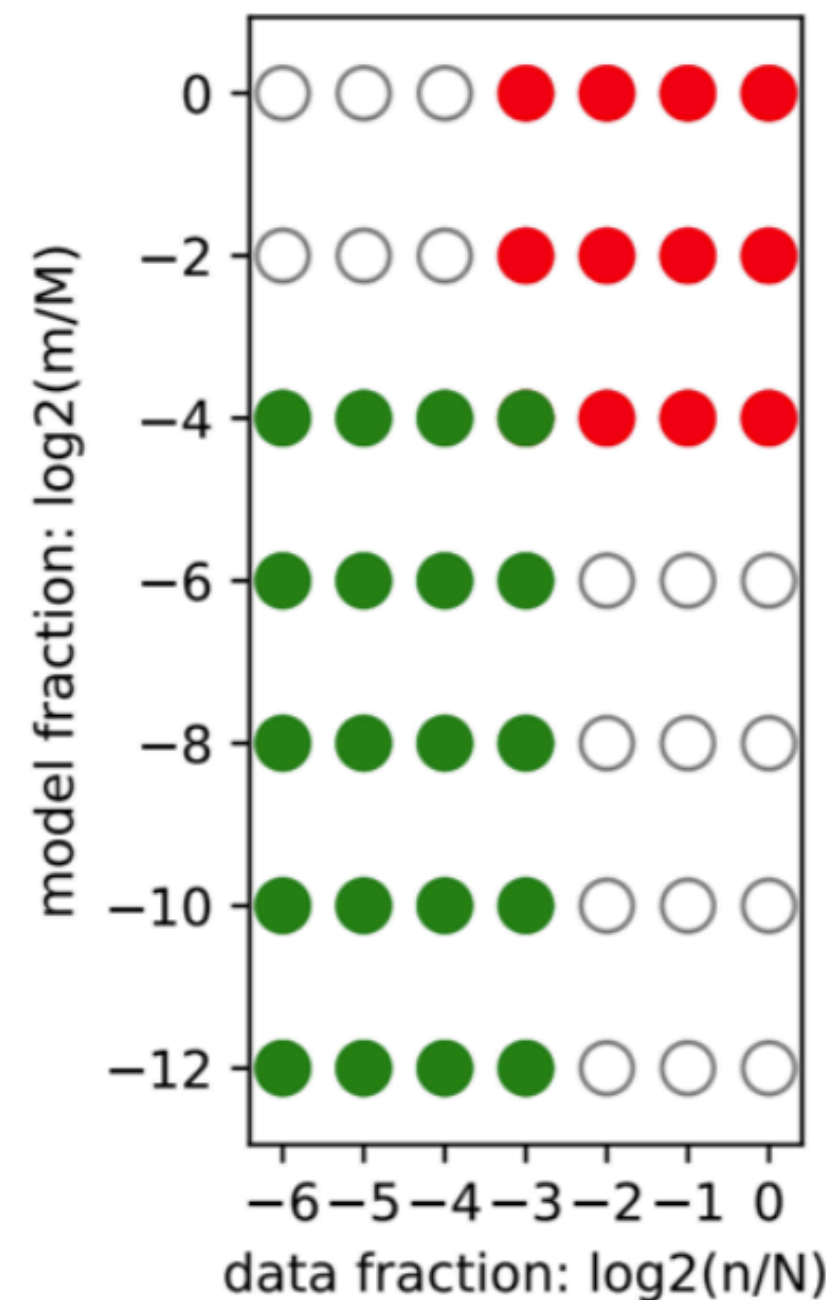
Provides surprisingly good fits to model-data joint error.



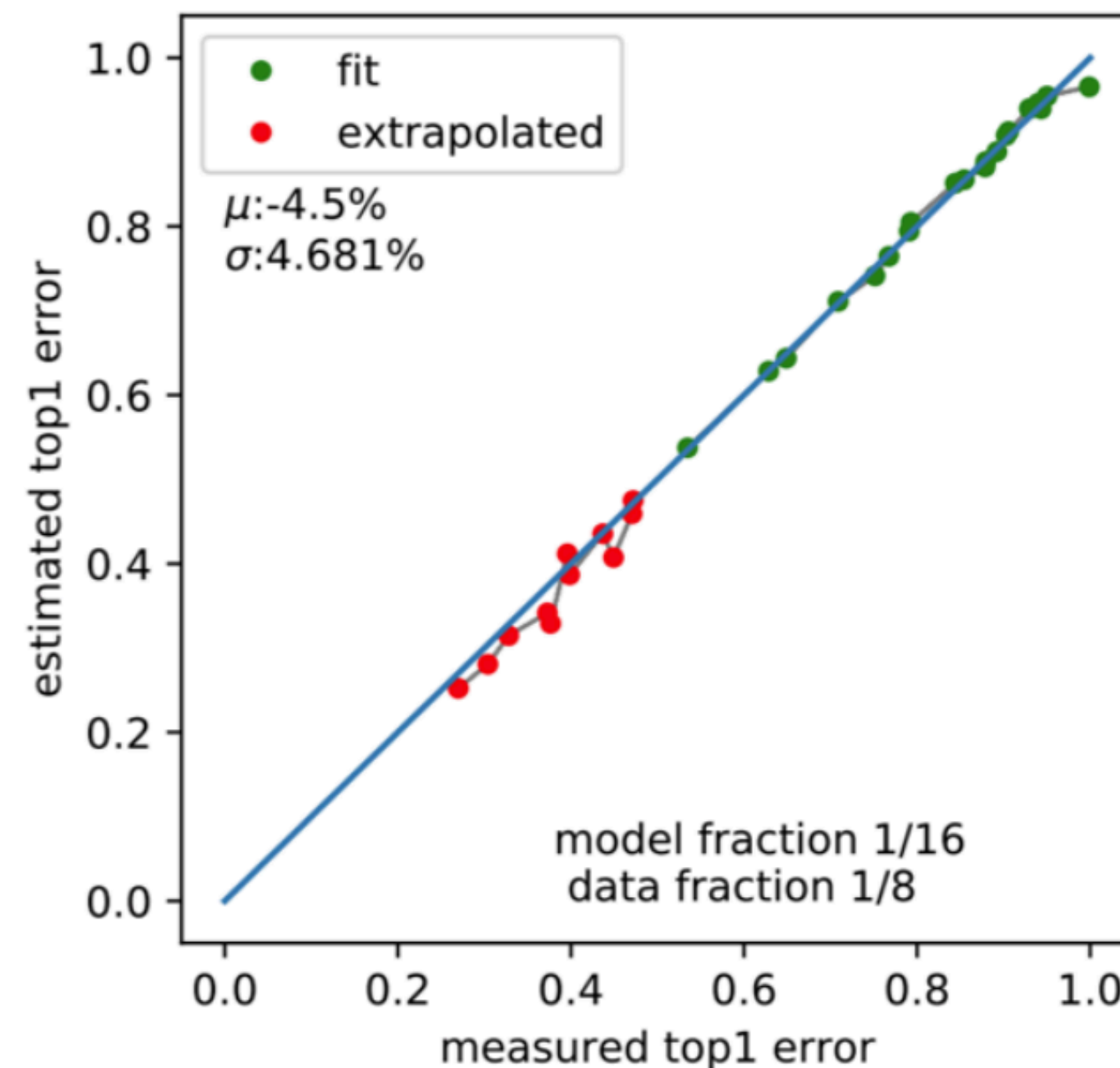
(a) Wiki103 error (cross entropy) landscape.

Model x Data Scaling

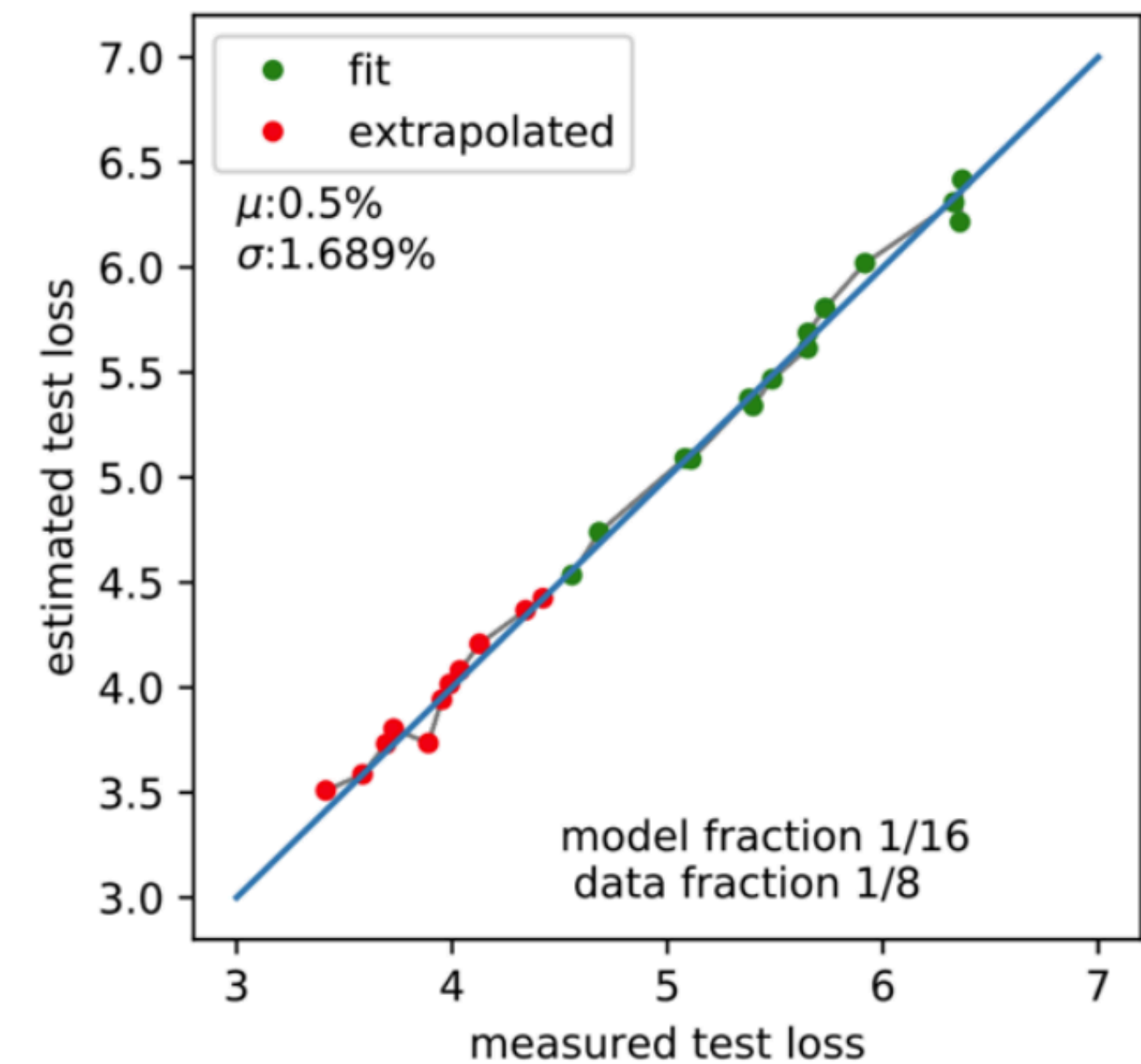
From Rosenfeld – fit scaling exponents on small data, small models. Predict rest.



(a) Illustration.



(b) Extrapolation on ImageNet



(c) Extrapolation on WikiText-103.

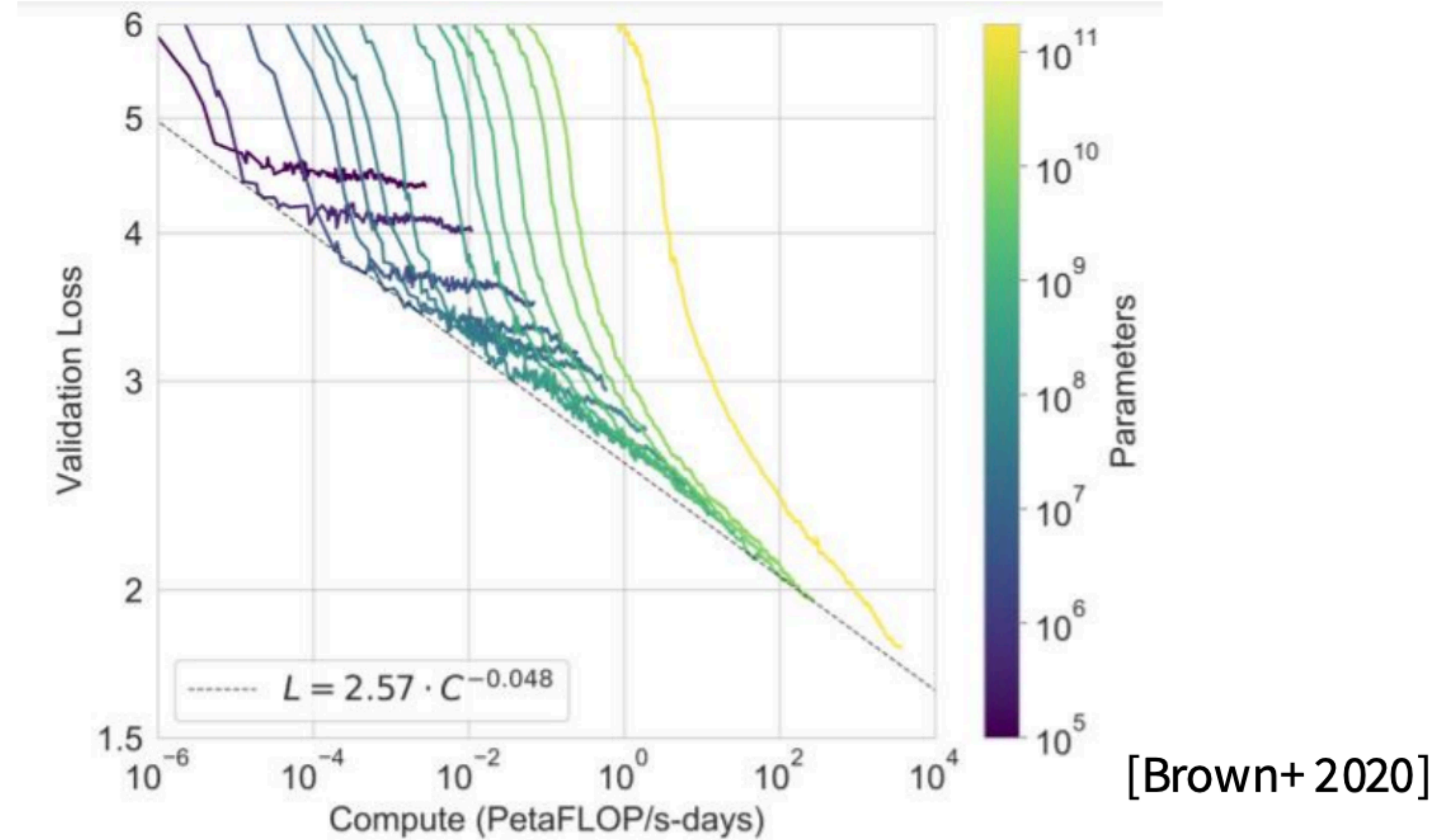
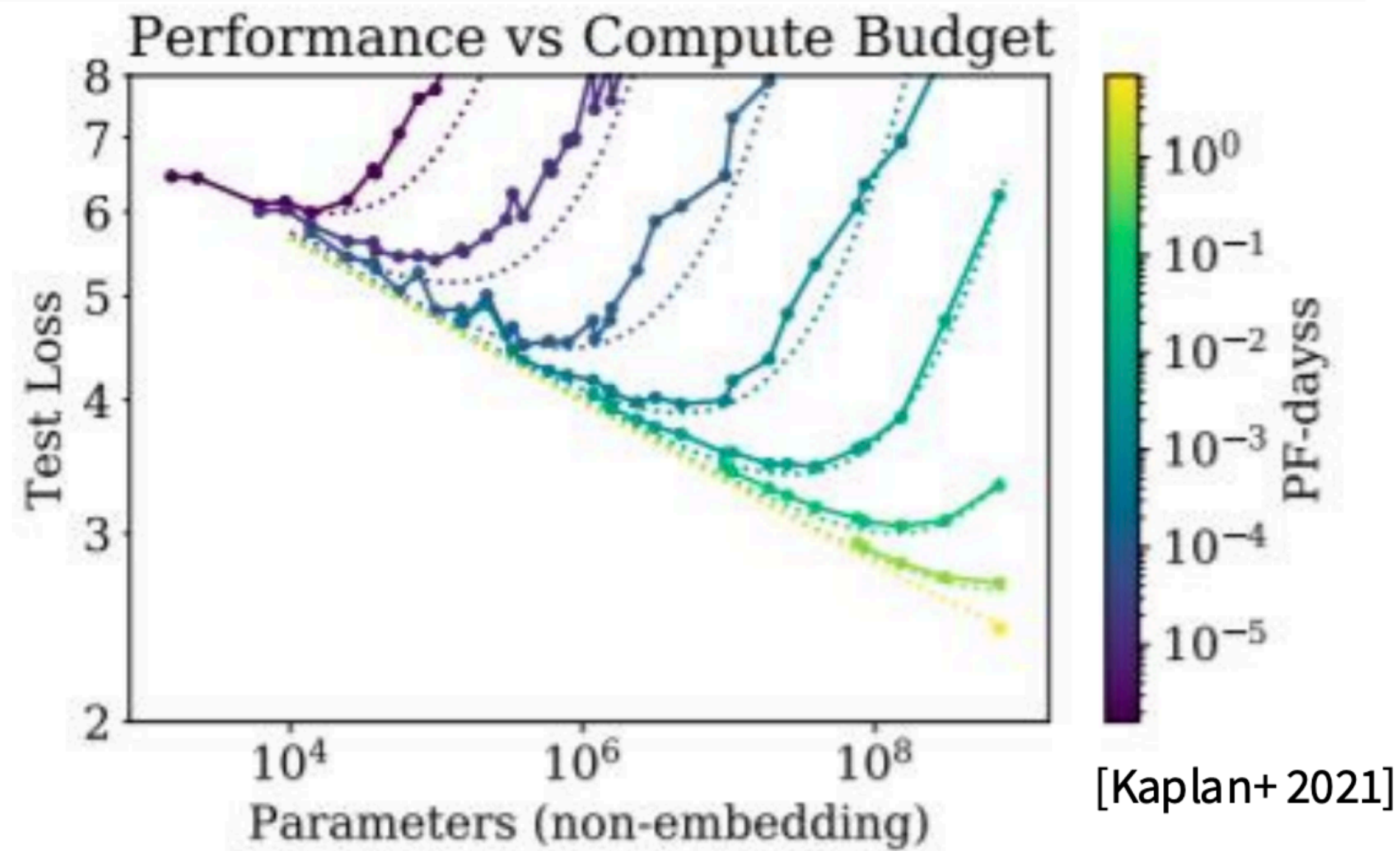
Trading off data size and model size: optimize $n^{-\alpha} + m^{-\beta} + C$ with your costs.

Compute Tradeoffs

Q: what about other resources? Compute vs performance?

For a fixed compute budget...

Big model that's undertrained vs small model that's well trained?

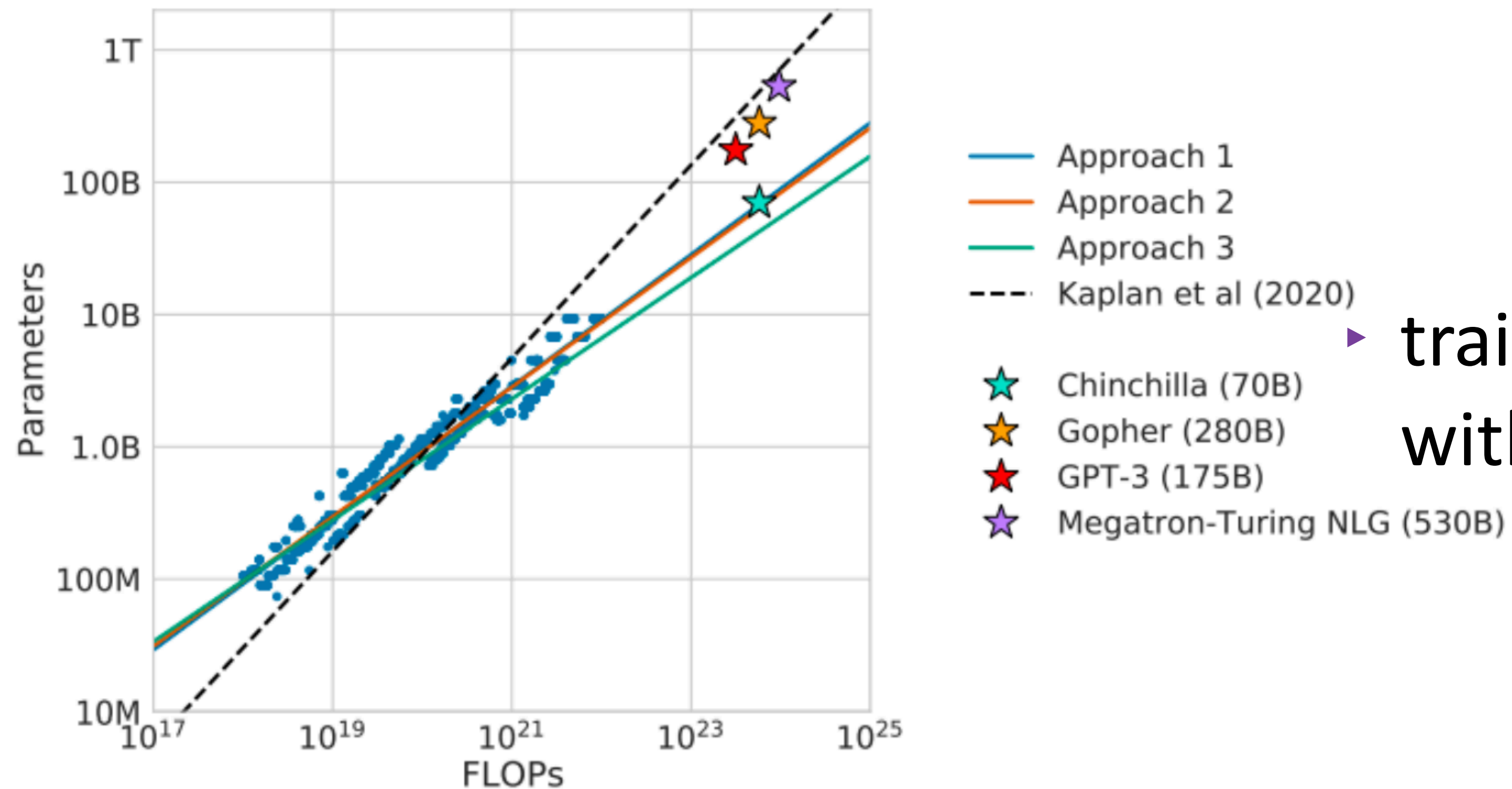


Scaling laws let us navigate this tradeoff

Compute Tradeoffs

Rosenfeld, Kaplan both predict relationship of data, model and perf.

Chinchilla [Hoffman et al] argue these fits are quite off.



► train on more data
with a smaller model

Compute Tradeoffs

Approach	Coeff. a where $N_{opt} \propto C^a$	Coeff. b where $D_{opt} \propto C^b$
1. Minimum over training curves	0.50 (0.488, 0.502)	0.50 (0.501, 0.512)
2. IsoFLOP profiles	0.49 (0.462, 0.534)	0.51 (0.483, 0.529)
3. Parametric modelling of the loss	0.46 (0.454, 0.455)	0.54 (0.542, 0.543)
Kaplan et al. (2020)	0.73	0.27

The chinchilla authors suggest 3 ways of fitting scaling laws – we’ll go over each.

They mostly (minus method 3) suggest similar constants. More on this later..

Method 1: Minimum over Runs

Similar to the FLOPS figure on Kaplan –
the minimum over the union of all training curves is a power law.

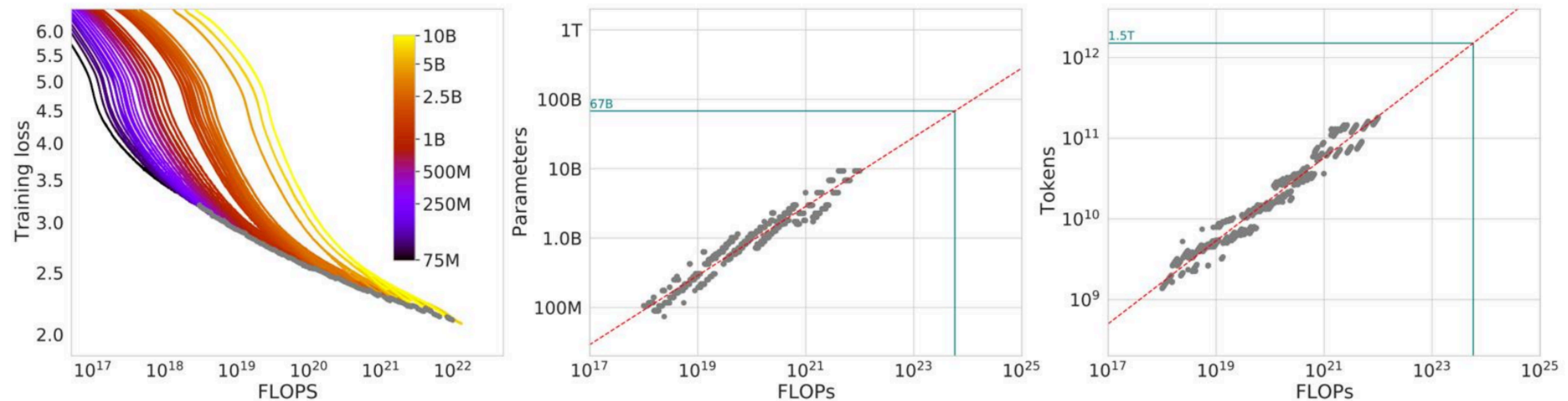


Figure 2 | **Training curve envelope.** On the **left** we show all of our different runs. We launched a range of model sizes going from 70M to 10B, each for four different cosine cycle lengths. From these curves, we extracted the envelope of minimal loss per FLOP, and we used these points to estimate the optimal model size (**center**) for a given compute budget and the optimal number of training tokens (**right**). In green, we show projections of optimal model size and training token count based on the number of FLOPs used to train *Gopher* (5.76×10^{23}).

Method 2: IsoFLOPs

Pick a range of FLOP budgets, vary the total parameter count, take the min over these convex shapes. The minima form a power law.

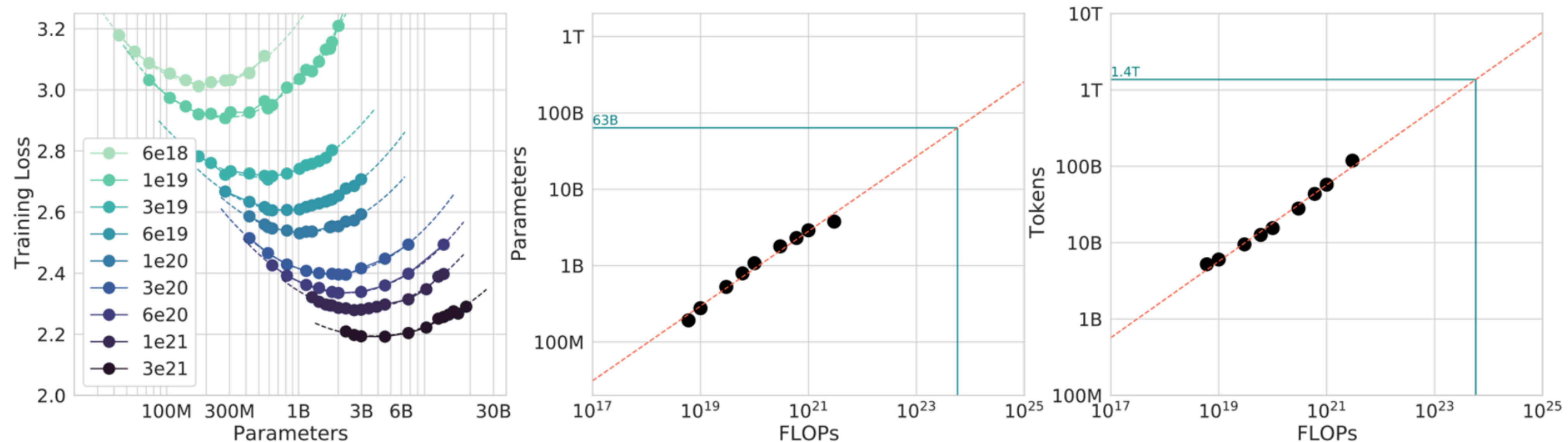


Figure 3 | IsoFLOP curves. For various model sizes, we choose the number of training tokens such that the final FLOPs is a constant. The cosine cycle length is set to match the target FLOP count. We find a clear valley in loss, meaning that for a given FLOP budget there is an optimal model to train (**left**). Using the location of these valleys, we project optimal model size and number of tokens for larger models (**center** and **right**). In green, we show the estimated number of parameters and tokens for an *optimal* model trained with the compute budget of *Gopher*.

Method 3: Joint Fits

Run a bunch of models on the size-data grid. Use least squares to fit a joint scaling law

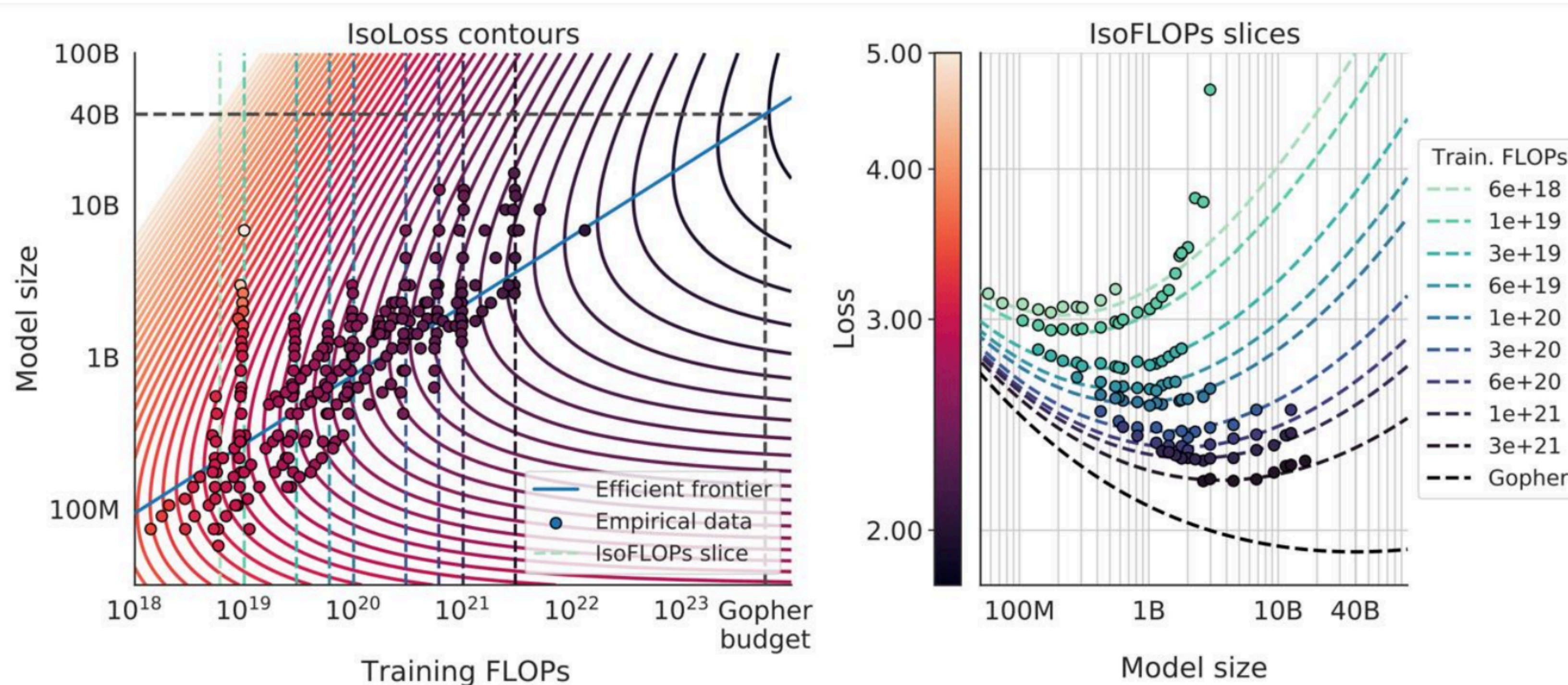


Figure 4 | **Parametric fit.** We fit a parametric modelling of the loss $\hat{L}(N, D)$ and display contour (**left**) and isoFLOP slices (**right**). For each isoFLOP slice, we include a corresponding dashed line in the left plot. In the left plot, we show the efficient frontier in blue, which is a line in log-log space. Specifically, the curve goes through each iso-loss contour at the point with the fewest FLOPs. We project the optimal model size given the *Gopher* FLOP budget to be 40B parameters.

Overtraining

Chinchilla aims to tell you what gives the best model for fixed training compute..

But most of the compute in a real deployment is inference.. So we should ‘over’ train

- **GPT3** – 2 tokens / param
- **Chinchilla** – 20 tokens / param
- **LLaMA65B** – 22 tokens / param
- **Llama 2 70B** – 29 tokens / param
- **Mistral 7B** – 110 tokens / param
- **Llama 3 70B** – 215 tokens / param

The more usage we expect, the more it becomes worth it to pay the upfront cost

Intro to Pre-training and Prompting

Data Scaling Laws

Model Scaling Laws

Chinchilla: Data x Model

Putting it together

Putting it together: DCLM and OLMo 3

Slide credit: Tatsu Hashimoto

DataComp Language Modeling

DataComp-LM: In search of the next generation of training sets for language models

Jeffrey Li^{*1,2} Alex Fang^{*1,2} Georgios Smyrnis^{*4} Maor Ivgi^{*5}
Matt Jordan⁴ Samir Gadre^{3,6} Hritik Bansal⁸ Etash Guha^{1,15} Sedrick Keh³ Kushal Arora³
Saurabh Garg¹³ Rui Xin¹ Niklas Muennighoff²² Reinhard Heckel¹² Jean Mercat³
Mayee Chen⁷ Suchin Gururangan¹ Mitchell Wortsman¹ Alon Albalak^{19,20} Yonatan Bitton¹⁴
Marianna Nezhurina^{9,10} Amro Abbas²³ Cheng-Yu Hsieh¹ Dhruva Ghosh¹ Josh Gardner¹
Maciej Kilian¹⁷ Hanlin Zhang¹⁸ Rulin Shao¹ Sarah Pratt¹ Sunny Sanyal⁴ Gabriel Ilharco¹
Giannis Daras⁴ Kalyani Marathe¹ Aaron Gokaslan¹⁶ Jieyu Zhang¹ Khyathi Chandu¹¹
Thao Nguyen¹ Igor Vasiljevic³ Sham Kakade¹⁸ Shuran Song^{6,7} Sujay Sanghavi⁴
Fartash Faghri² Sewoong Oh¹ Luke Zettlemoyer¹ Kyle Lo¹¹ Alaaeldin El-Nouby²
Hadi Pouransari² Alexander Toshev² Stephanie Wang¹ Dirk Groeneveld¹¹ Luca Soldaini¹¹
Pang Wei Koh¹ Jenia Jitsev^{9,10} Thomas Kollar³ Alexandros G. Dimakis^{4,21}
Yair Carmon⁵ Achal Dave^{†3} Ludwig Schmidt^{†1,7} Vaishal Shankar^{†2}

¹University of Washington, ²Apple, ³Toyota Research Institute, ⁴UT Austin,
⁵Tel Aviv University, ⁶Columbia University, ⁷Stanford, ⁸UCLA, ⁹JSC, ¹⁰LAION, ¹¹AI2,
¹²TUM, ¹³CMU, ¹⁴Hebrew University, ¹⁵SambaNova, ¹⁶Cornell, ¹⁷USC, ¹⁸Harvard,
¹⁹UCSB, ²⁰SynthLabs, ²¹Bespokelabs.AI, ²²Contextual AI, ²³DatologyAI

DataComp Language Modeling

Followed on earlier “DataComp” projects for language modeling

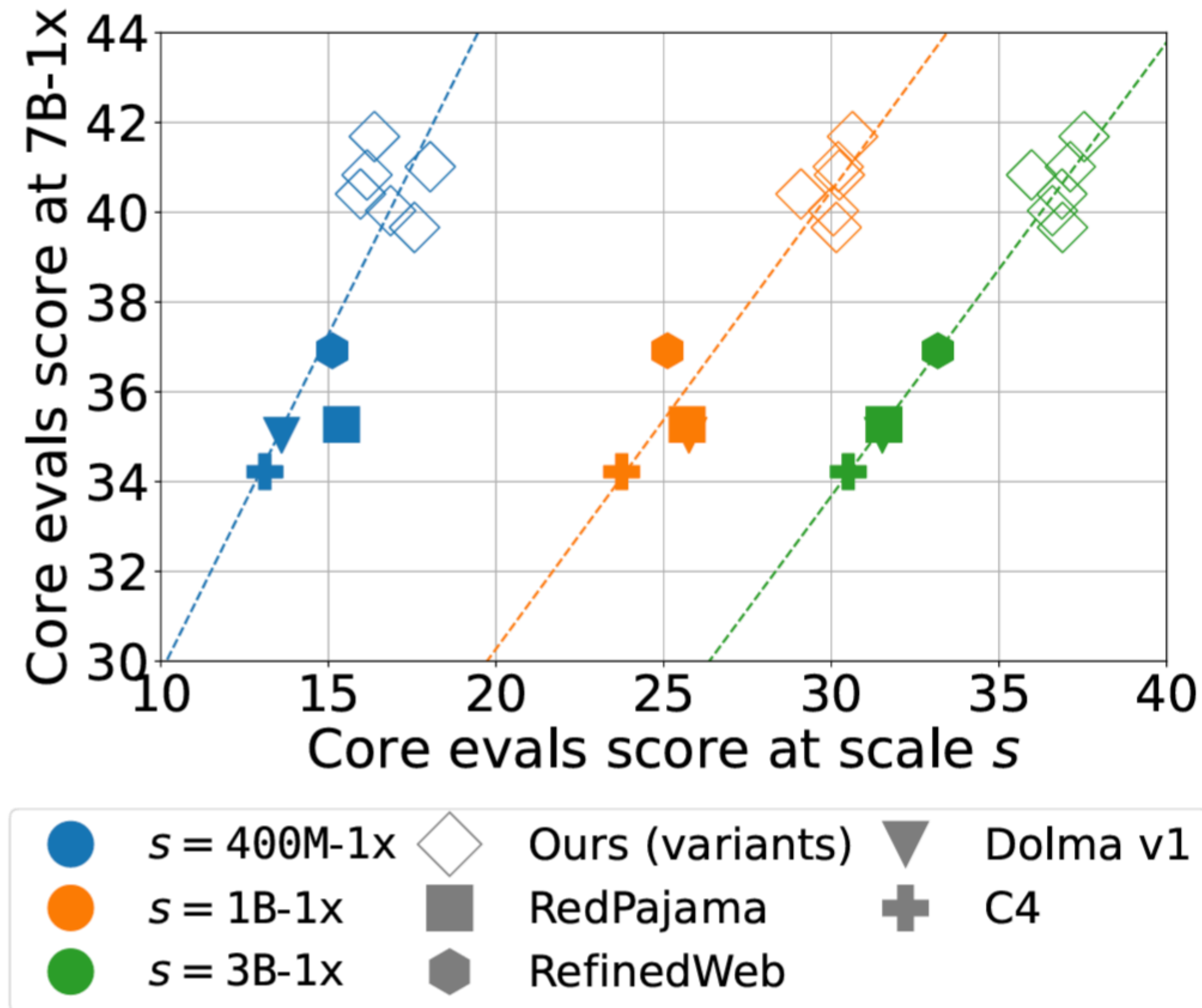
Fixed dataset to allow for testing of algorithms with data held constant

Scale	Model parameters	Train tokens	Train FLOPs	Train H100 hours	Pool size
400M-1x	412M	8.2B	2.0e19	26	469B
1B-1x	1.4B	28.8B	2.4e20	240	1.64T
3B-1x	2.8B	55.9B	9.4e20	740	3.18T
7B-1x	6.9B	138B	5.7e21	3,700	7.85T
7B-2x	6.9B	276B	1.1e22	7,300	15.7T

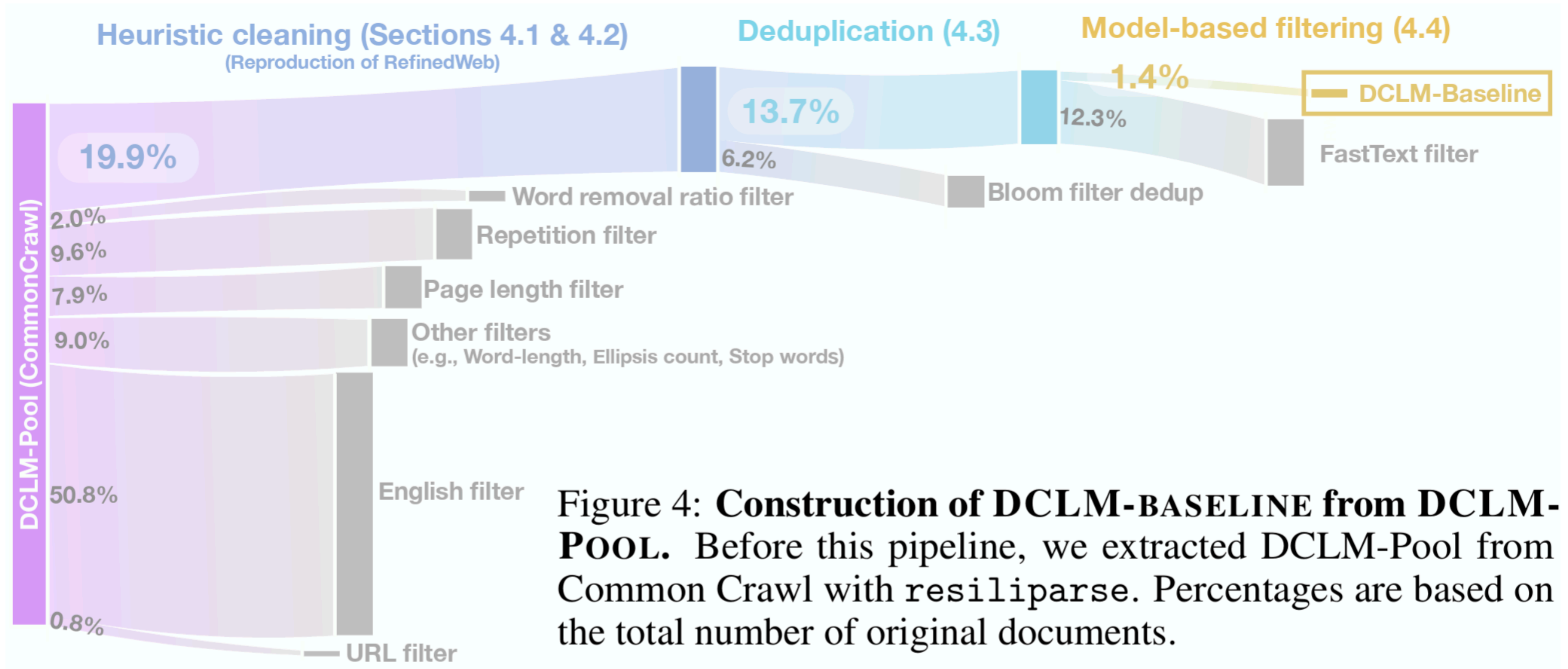
...but what people ended up using was actually just the dataset they made

Predictability

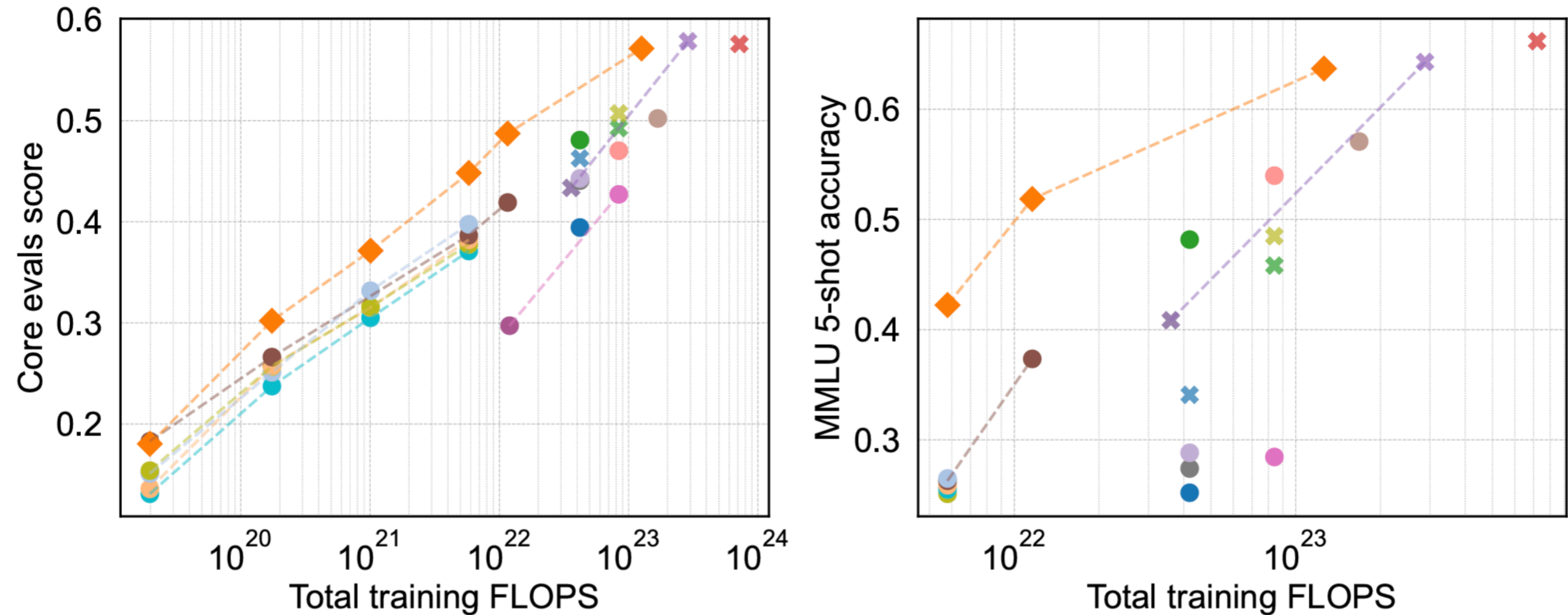
Evals at small scale predict scores at higher scale (positive correlation)



Data Filtering



Data Filtering



Olmo 3

Olmo Team★

Allyson Ettinger♥¹ Amanda Bertsch♥^{1,3} Bailey Kuehl♥¹ David Graham♥¹
David Heineman♥¹ Dirk Groeneveld♥¹ Faeze Brahman♥¹ Finbarr Timbers♥¹
Hamish Ivison♥^{1,2} Jacob Morrison♥^{1,2} Jake Poznanski♥¹ Kyle Lo♥^{1,2} Luca Soldaini♥¹
Matt Jordan♥¹ Mayee Chen♥^{1,4} Michael Noukhovitch♥^{1,5,6} Nathan Lambert♥¹
Pete Walsh♥¹ Pradeep Dasigi♥¹ Robert Berry♥¹ Saumya Malik♥¹ Saurabh Shah♥¹
Scott Geng♥^{1,2} Shane Arora♥¹ Shashank Gupta♥¹ Taira Anderson♥¹ Teng Xiao♥¹
Tyler Murray♥¹ Tyler Romero♥¹ Victoria Graf♥^{1,2}

Akari Asai^{1,3} Akshita Bhagia¹ Alexander Wettig⁷ Alisa Liu² Aman Rangapur¹
Chloe Anastasiades¹ Costa Huang¹ Dustin Schwenk¹ Harsh Trivedi¹ Ian Magnusson^{1,2}
Jaron Lochner¹ Jiacheng Liu¹ Lester James V. Miranda¹ Maarten Sap^{1,3} Malia Morgan¹
Michael Schmitz¹ Michal Guerquin¹ Michael Wilson¹ Regan Huff¹ Ronan Le Bras¹
Rui Xin² Rulin Shao² Sam Skjonsberg¹ Shannon Zejiang Shen⁸ Shuyue Stella Li²
Tucker Wilde¹ Valentina Pyatkin¹ Will Merrill¹ Yapei Chang⁹ Yuling Gu¹ Zhiyuan Zeng^{1,2}

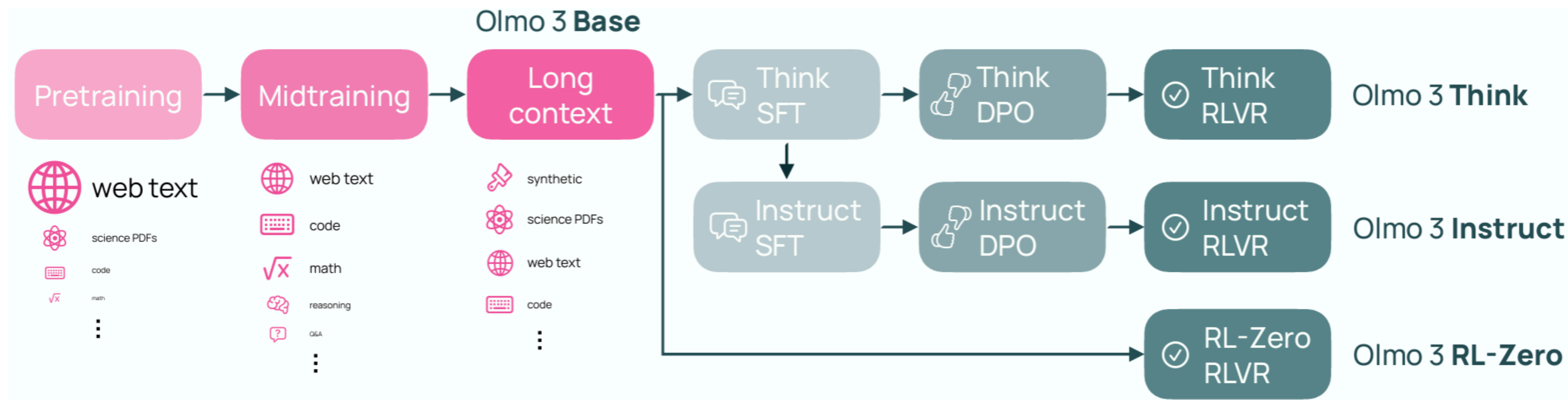
Ashish Sabharwal¹ Luke Zettlemoyer² Pang Wei Koh^{1,2}
Ali Farhadi^{1,2} Noah A. Smith♥^{1,2} Hannaneh Hajishirzi♥^{1,2}

¹Allen Institute for AI ²University of Washington ³Carnegie Mellon University ⁴Stanford University ⁵Mila
⁶Université de Montréal ⁷Princeton University ⁸Massachusetts Institute of Technology ⁹University of Maryland

OLMo 3

Along with DCLM, one of the few “fully open” language models: open weights, open data, ability to reproduce given the compute.
(Note: Llama/Qwen/etc. lack open data.)

Several models; we will cover what all this means more with post-training:



OLMo 3

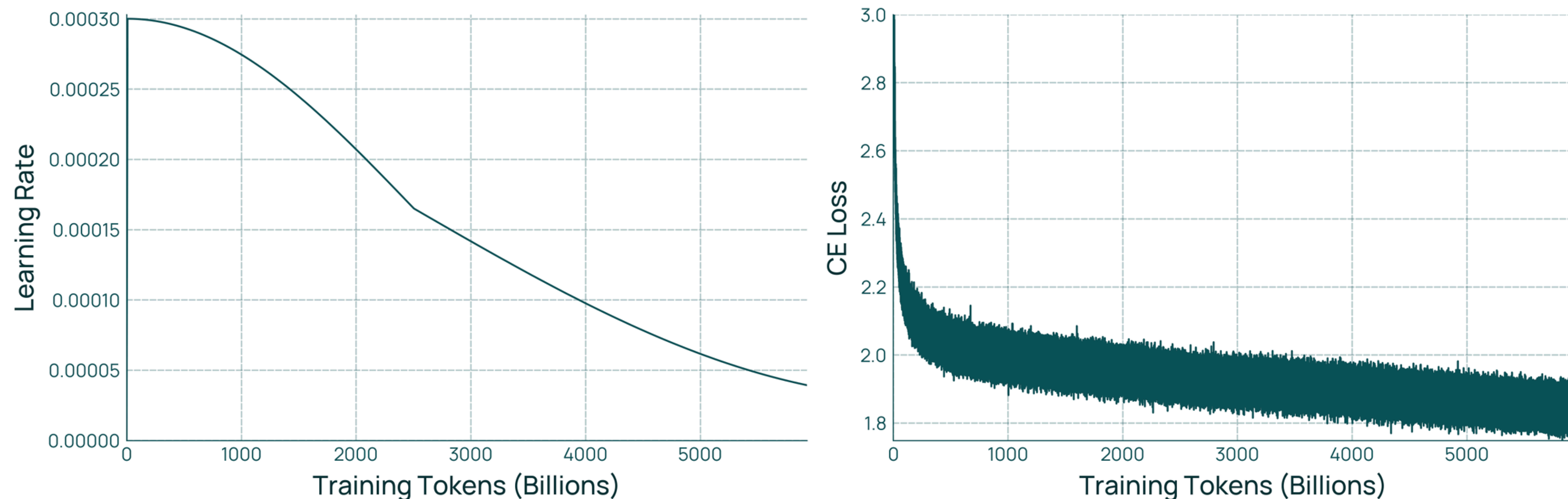


Figure 3 Learning rate schedule and loss for Olmo 3 Base 7B. The first half of the learning rate schedule is a cosine schedule over 5T tokens. We stretch the second half of the schedule to reach a target length of one epoch (5.93T tokens). Warm-up is 2000 steps, the peak learning rate is 3×10^{-4} , and the final learning rate is 10% of the peak LR.

Loss decreases very regularly here.

Administrative details and recap

Hyperparameter Optimization

Scaling Law History

Data Scaling Laws

Model Scaling Laws

Chinchilla: Data x Model

Putting it together